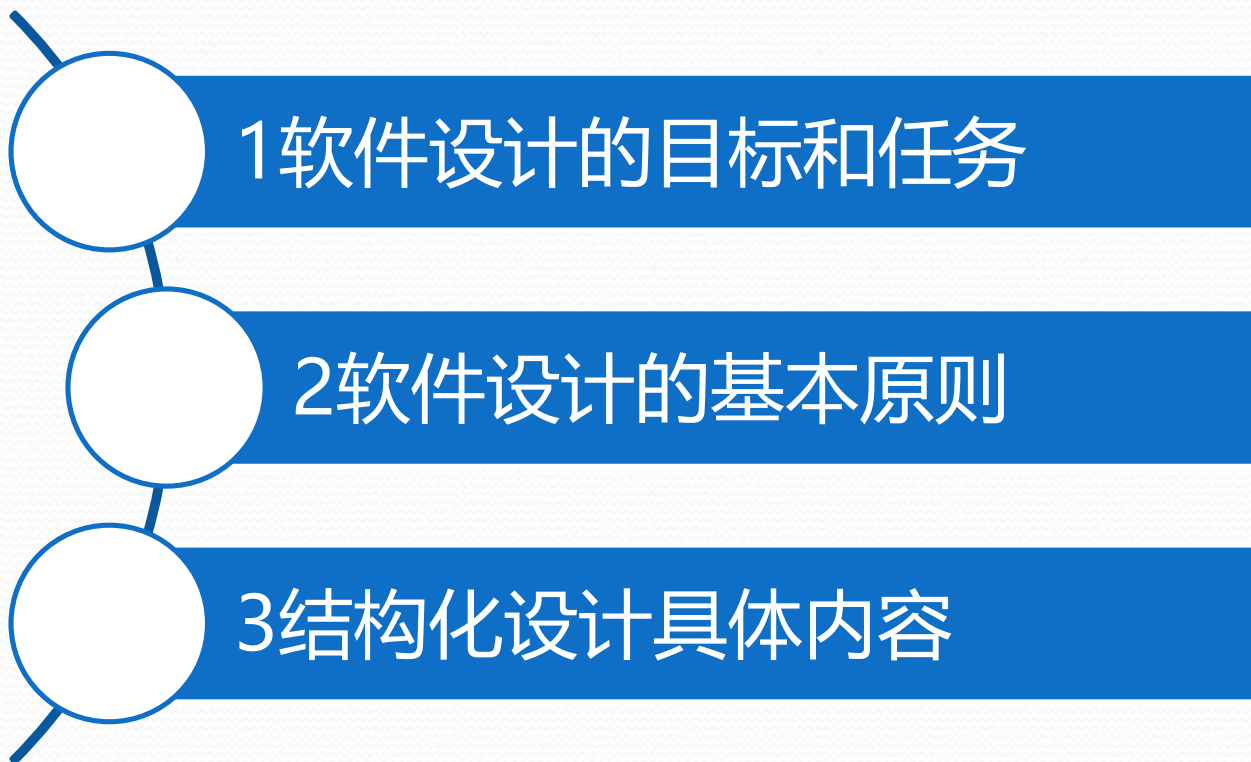


软件工程导论

杜莹

计算机与信息工程学院 数据科学系

面向过程（结构化）的软件设计



1 软件设计的目标和任务

- 1.1 软件设计的目标
- 1.2 软件设计的任务

1.1软件设计的目标

阶段	回答的中心问题		成果
系统分析	“做什么”	明确系统功能	逻辑模型
系统设计	“怎么做”	如何实现系统说明书规定的系统功能	物理模型

1.2 结构化软件设计的任务

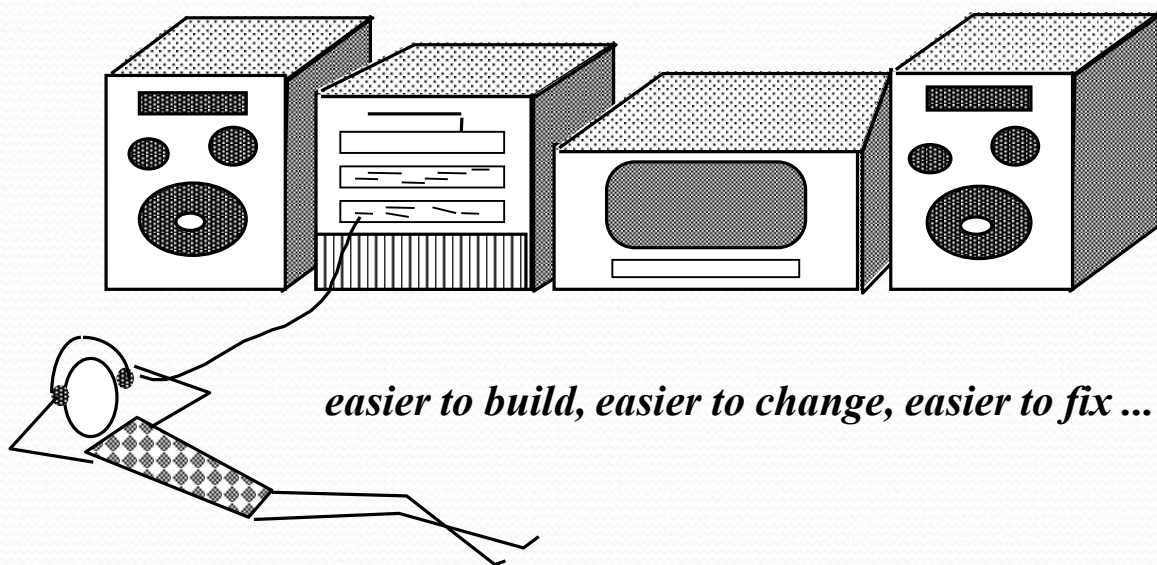
体系结构设计	定义软件模块间的关系	概要设计
接口设计	软件内部、外部及与人之间的通信	
数据设计	软件数据结构	详细设计
过程设计	软件组件的过程性描述	

2 软件设计的基本原则

- 2.1.模块化
- 2.2.抽象
- 2.3.逐步求精
- 2.4.信息隐蔽

2.1 模块化

- **模块**是组成目标系统逻辑模型和物理模型的基本单位，一般是一组程序语句：可通过名字访问的过程、函数、子程序或宏调用。
- **特点**：可以组合、分解和更换。

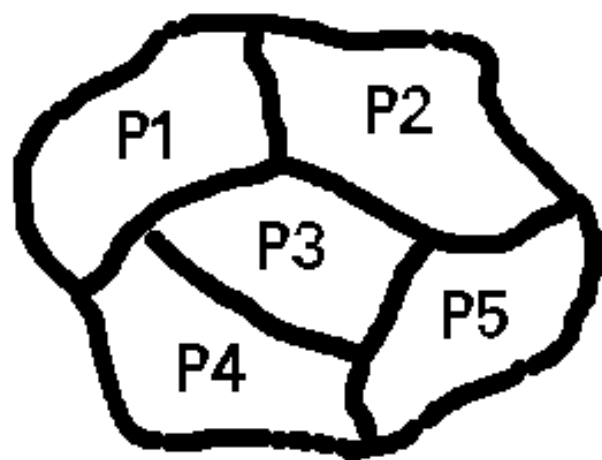


2.1 模块化

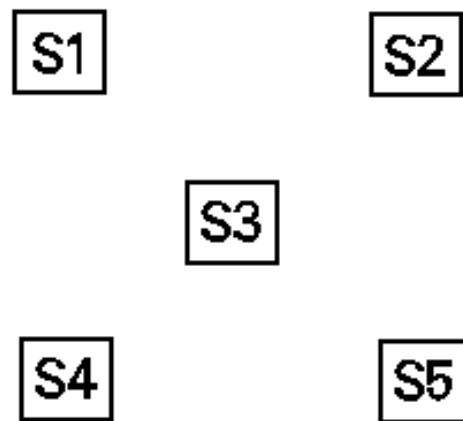
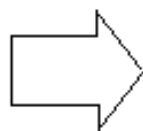
- 模块具有如下4个基本属性：
 - (1) **接口**：指模块的输入与输出。（外部属性）
 - (2) **功能**：指该模块实现什么功能，做什么事情。
 - (3) **逻辑**：描述内部如何实现要求的功能及所需的数据。
(内部数据、程序代码、内部属性)
 - (4) **状态**：该模块使用时的环境和条件，即模块的调用与被调用关系。

2.1 模块化

- **模块化**：解决一个复杂问题时，自顶向下逐层把软件划分成若干模块的过程
- 目的：为了降低软件的复杂性



需要通过软件解决的“问题”



软件的“解决方案”

2.1 模块化

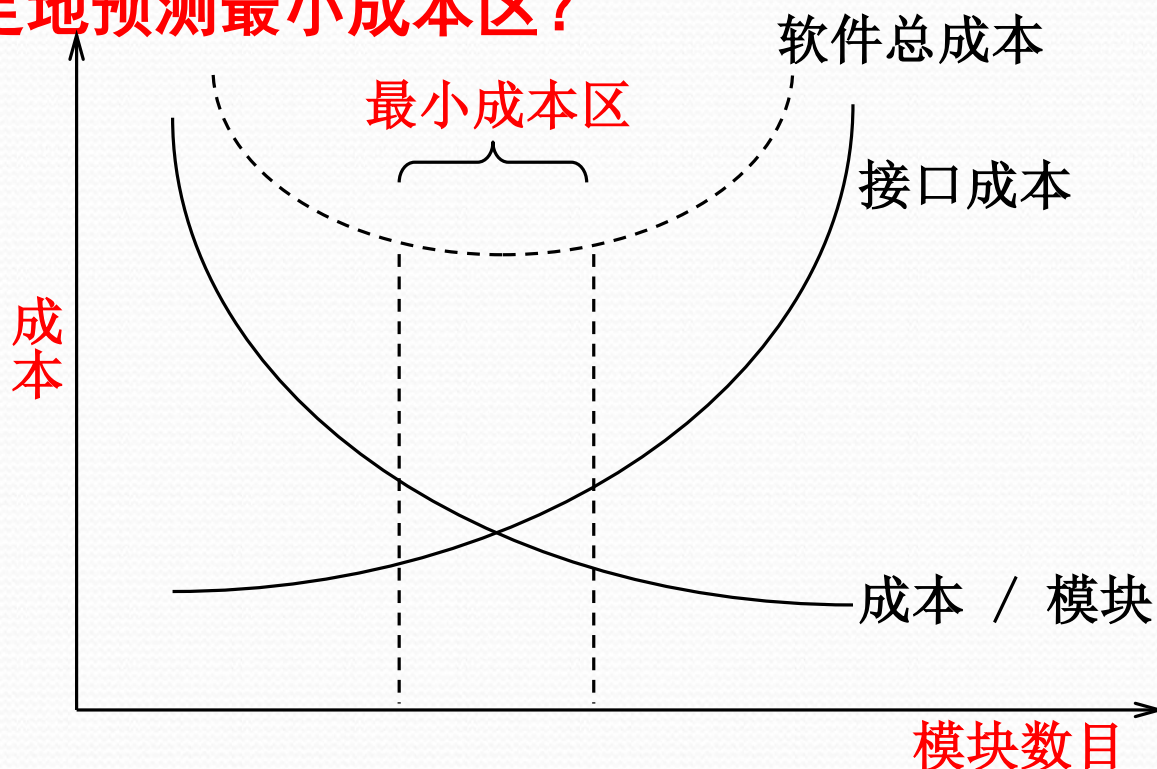
模块化论据：

- $C(x)$ 定义为问题 x 的复杂性
- $E(x)$ 定义为解决问题 x 所需要的工作量
- 对 $p1$ 和 $p2$ 两个问题，
若 $C(p1) > C(p2)$ ，则 $E(p1) > E(p2)$
- $C(p1 + p2) > C(p1) + C(p2)$
- $E(p1 + p2) > E(p1) + E(p2)$
- 不要过度模块化！每个模块的简单性将被集成的复杂性所掩盖。

2.1 模块化

- 模块化和软件成本

- 如何确定地预测最小成本区？



2.1 模块化

❖ 模块化的作用

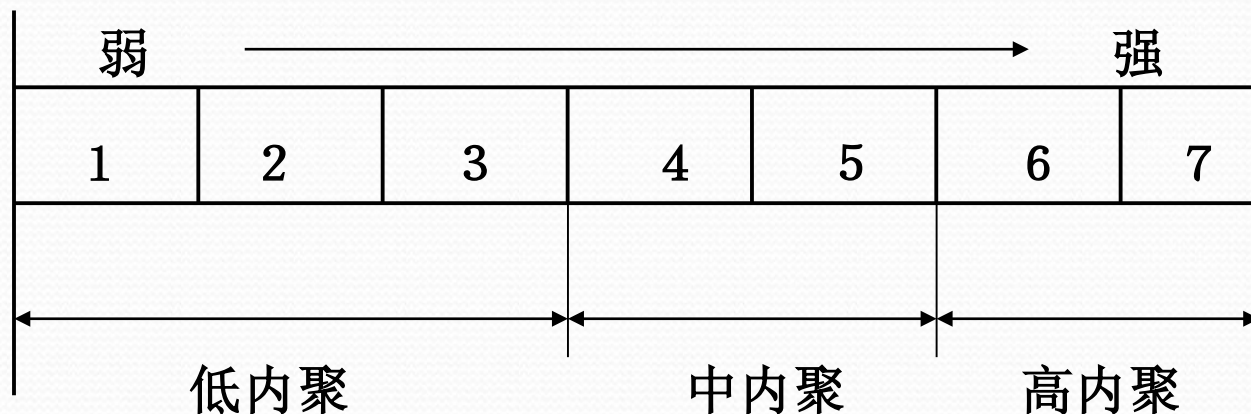
- **结构清晰**：容易设计也容易阅读和理解；
- **可靠性高**：程序错误通常局限在有关的模块及它们之间的接口中；
- **可修改性**：变动往往只涉及少数几个模块；
- **降低复杂度、便于分工**：复杂的大型程序可以由许多程序员分工编写不同的模块。

模块化的目标：模块独立性

- 模块独立性概括了系统分解时要遵守的准则
- 衡量模块独立性的两个准则
 - 内聚：模块内部各成分之间的联系，也称块内联系或模块强度
 - 耦合：一个模块与其它模块之间的联系，也称为块间联系
- 模块的独立性高
 - 块内联系强
 - 块间联系弱

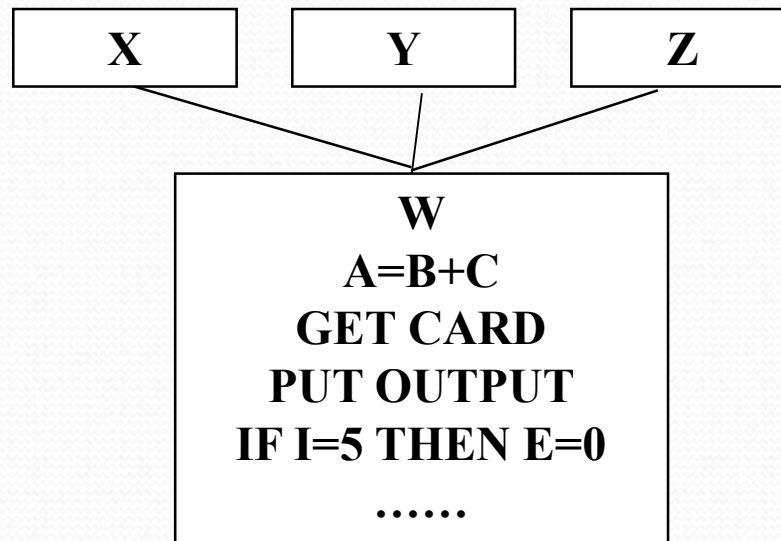
七种内聚情况

- 1.偶然性内聚
- 2.逻辑性内聚
- 3.时间性内聚
- 4.过程性内聚
- 5.通信性内聚
- 6.顺序性内聚
- 7.功能性内聚



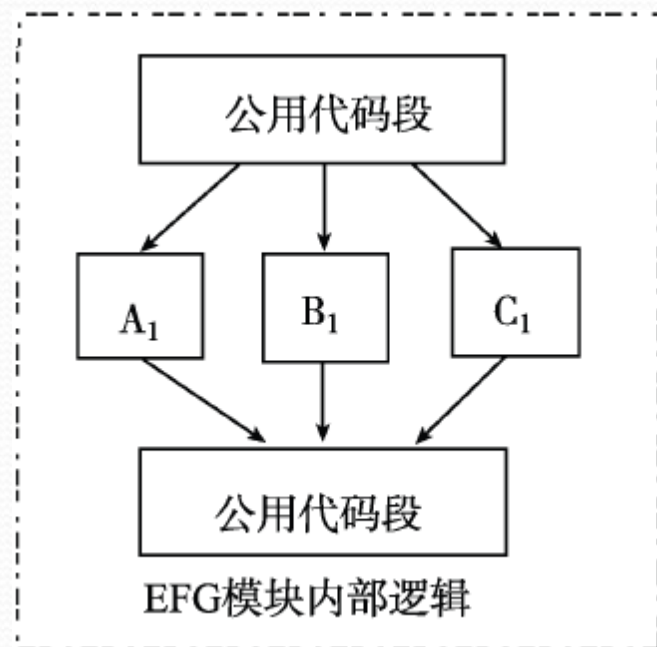
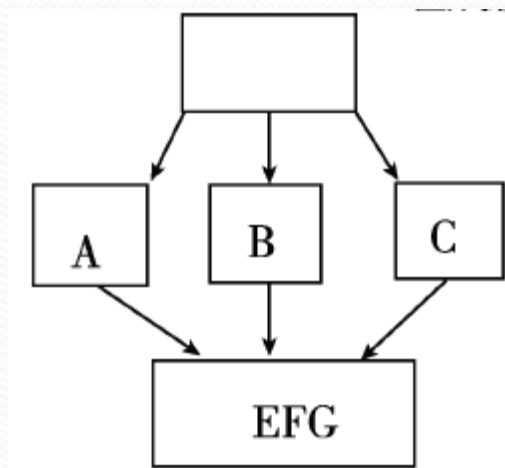
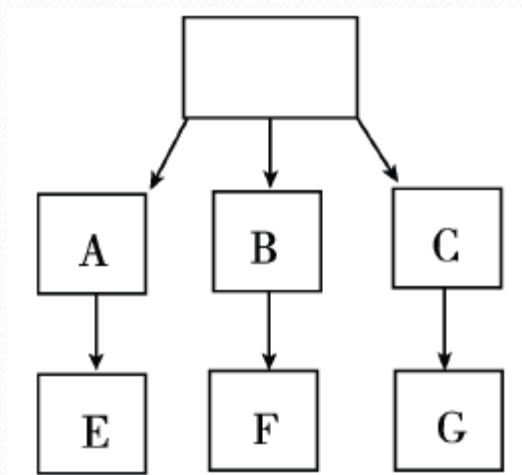
1-偶然性内聚

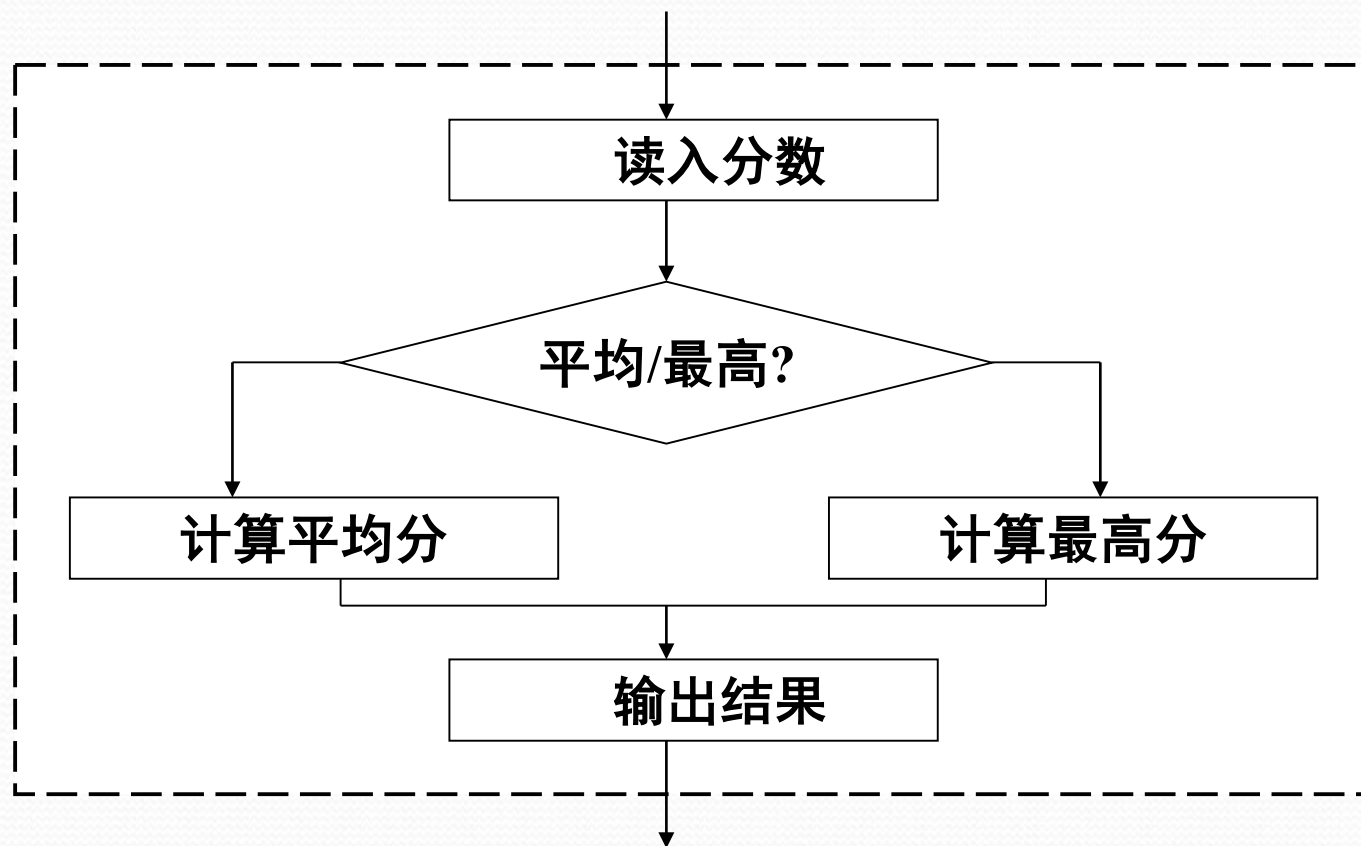
- 块内各组成成分在功能上是互不相关的。



2-逻辑性内聚

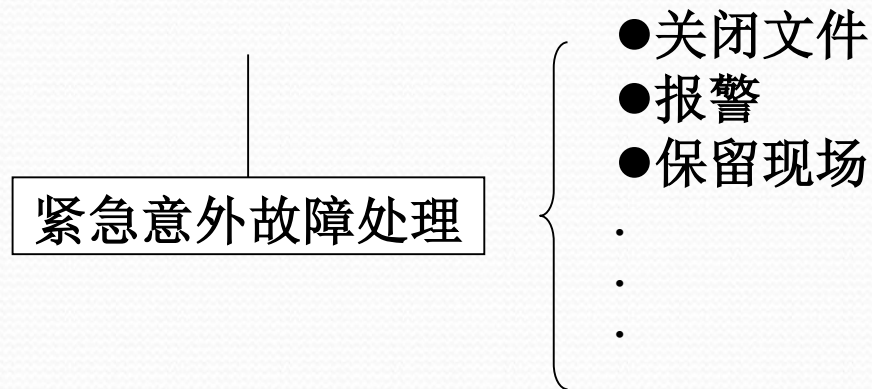
- 把几种相关、相似功能组合在一起，每次被调用时，由传递给模块的参数来确定该模块应完成哪一种功能。





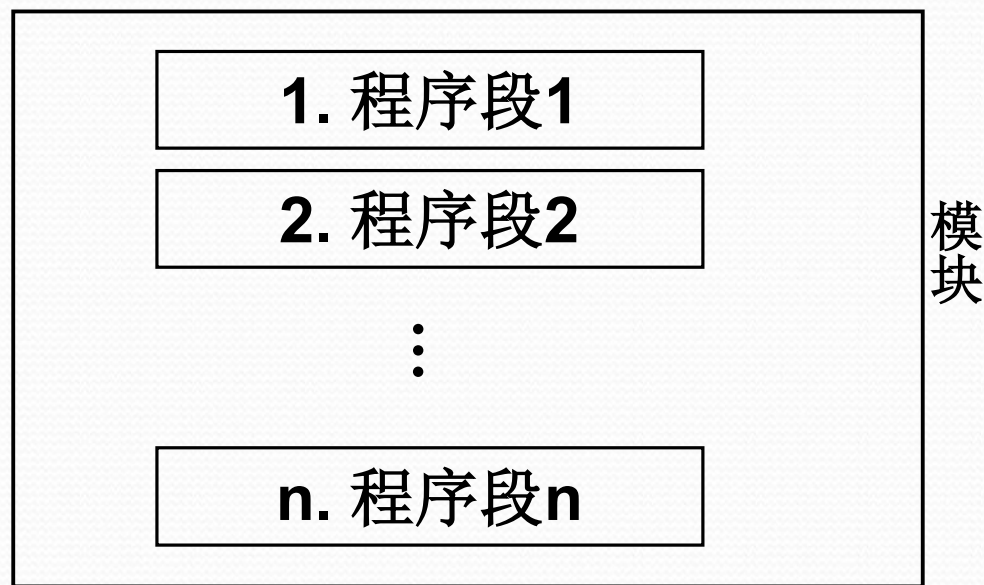
3-时间性内聚

- 如果一个模块所包含的任务必须在同一“时间”内完成，则这个模块的块内联系称为时间性内聚。



4-过程化内聚

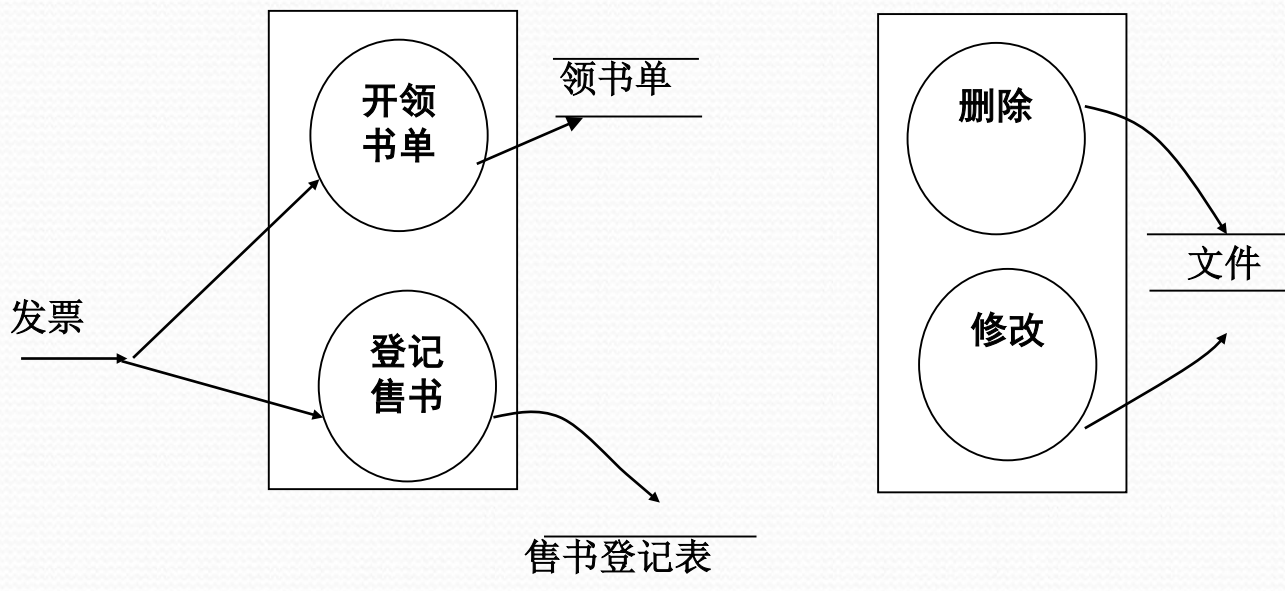
- 当一个模块中包含的一组任务必需按照某一特定的次序执行时，就称为过程性内聚模块。



每一程序段的
先后执行顺序
有严格要求，
必须依次完成。

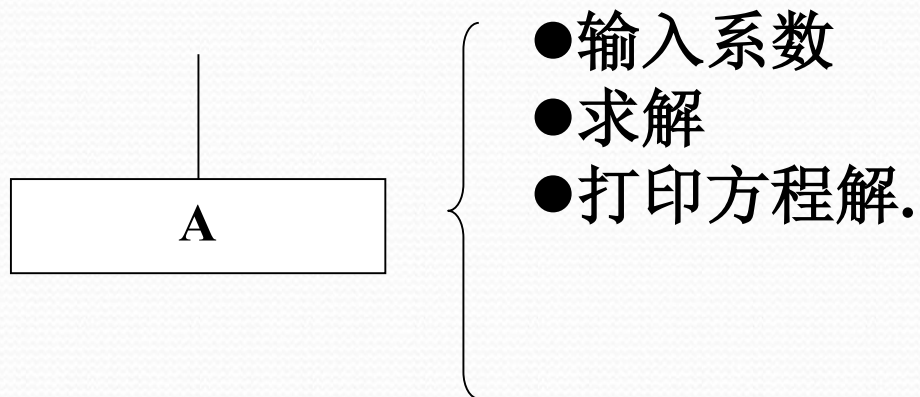
5-通信性内聚

- 模块内部的各个成分都使用同一种输入数据，或者产生同一个输出数据。它们靠公用数据而联系在一起，故称为通信性内聚。



6-顺序性内聚

- 如果一个模块内的处理元素和同一个功能密切相关，而且这些处理必须顺序执行，通常一个处理元素的输出数据作为下一个处理元素的输入数据，则成为顺序性内聚。



7-功能性内聚

- 所有成分结合在一起，用于完成一个单一的功能
- “一个模块一个功能” —— 模块化设计的准则

耦合

- 1.非直接耦合 2.数据耦合
- 3.特征耦合 4.控制耦合
- 5.外部耦合 6.公共耦合
- 7.内容耦合

低 ————— 耦合性 —————→ 高

非直接耦合	数据耦合	标记耦合	控制耦合	外部耦合	公共耦合	内容耦合
-------	------	------	------	------	------	------

强 ←———— 模块独立性 —————→ 弱

1-内容耦合

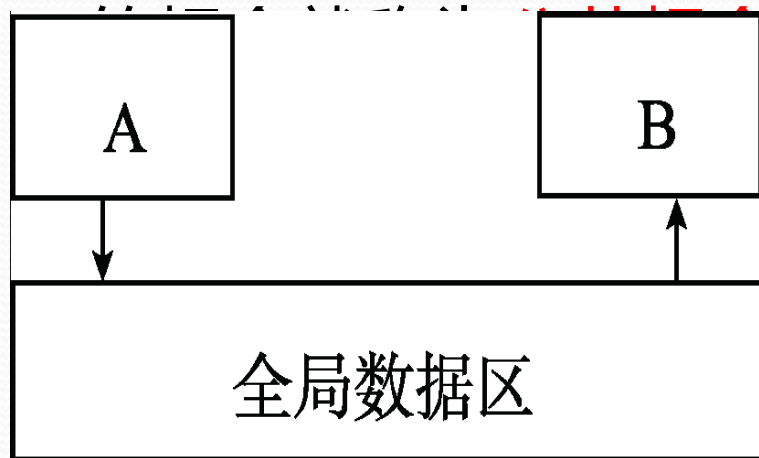
- 如果一个模块直接访问另一个模块的内部数据;
- 或者一个模块不通过正常入口转到另一模块内部;
- 或者两个模块有一部分程序代码重叠;
- 或者一个模块有多个入口, 则两个模块之间就发生了内容耦合。

2-公共耦合

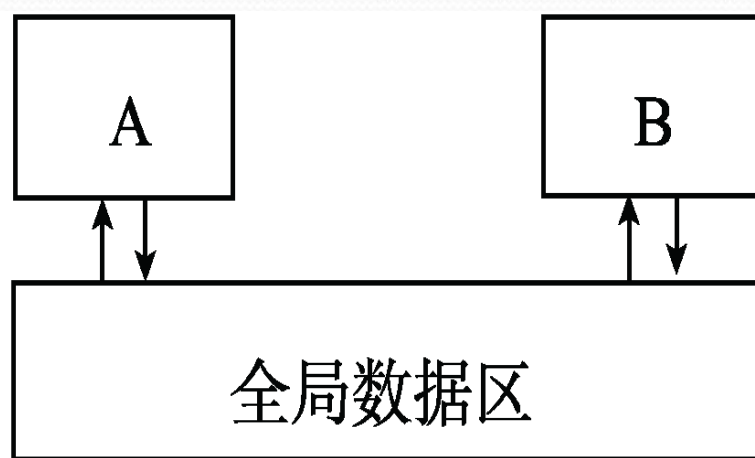
- 若一组模块都访问同一个公共数据环境，则它们之间的耦合就称为公共耦合。
- 公共的数据环境可以是全局数据区、共享的通信区、内存的公共覆盖区、任何介质上的文件、物理设备等。

2-公共耦合

- 若一组模块都访问同一个公共数据环境，则它们之间



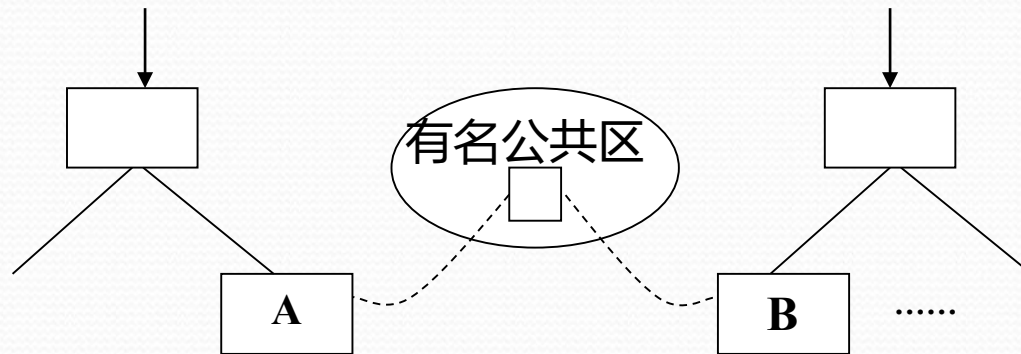
(a) 松散의公共耦合



(b) 紧密的公共耦合

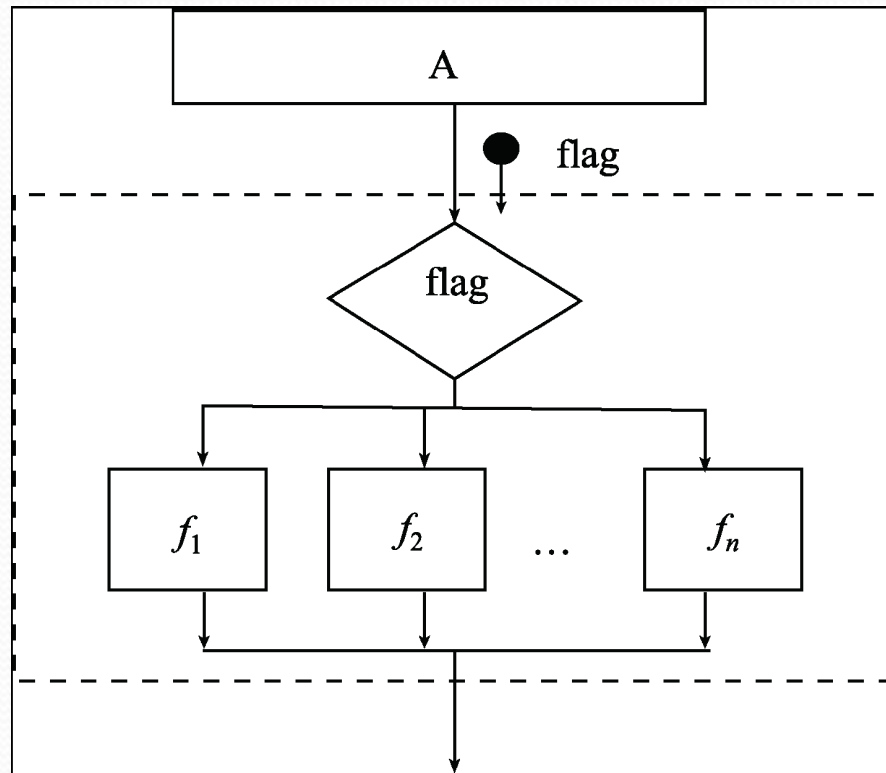
3-外部耦合

- 若允许一组模块访问同一个**全局变量**，可称它们为**外部耦合**。
- 外部耦合与公共耦合的区别在于在外部耦合中不存在依赖于一个数据结构内部各项的物理安排。



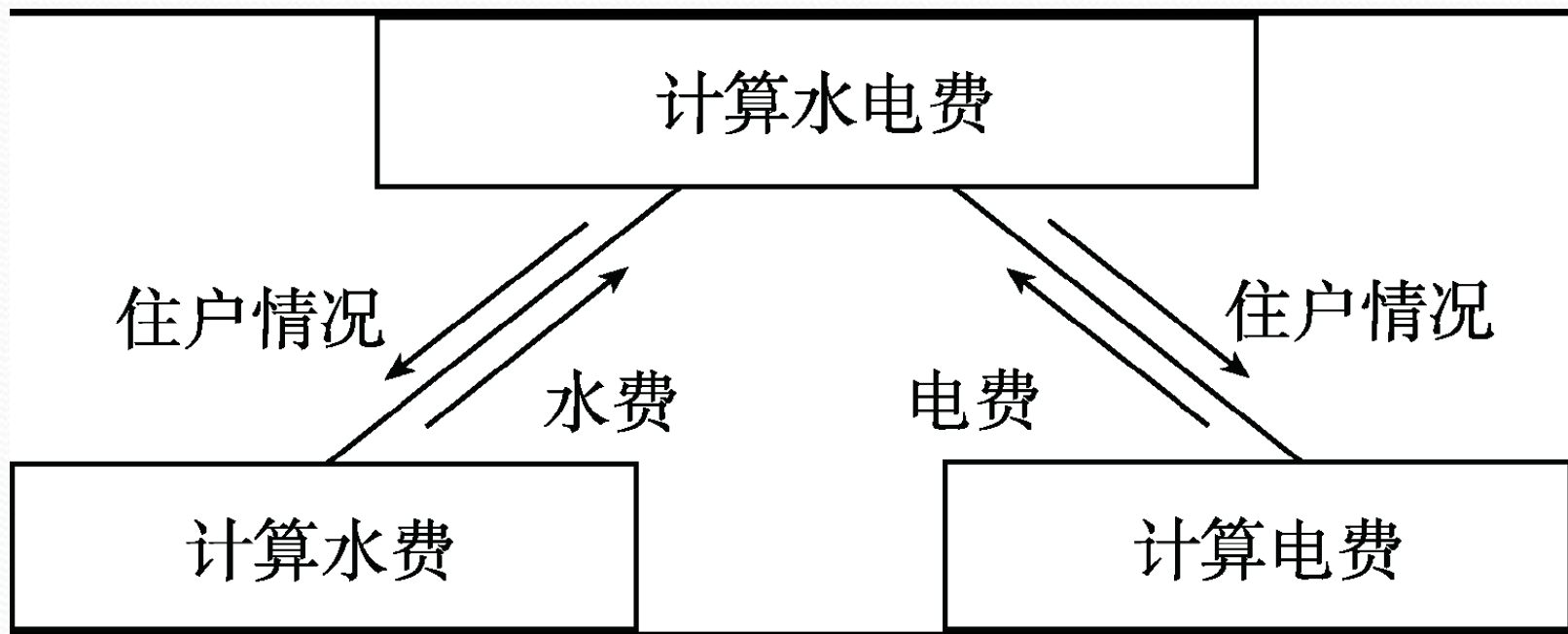
4-控制耦合

- 如果一个模块通过传送开关、标志、名字等控制信息，明显地控制选择另一模块的功能，就是控制耦合。



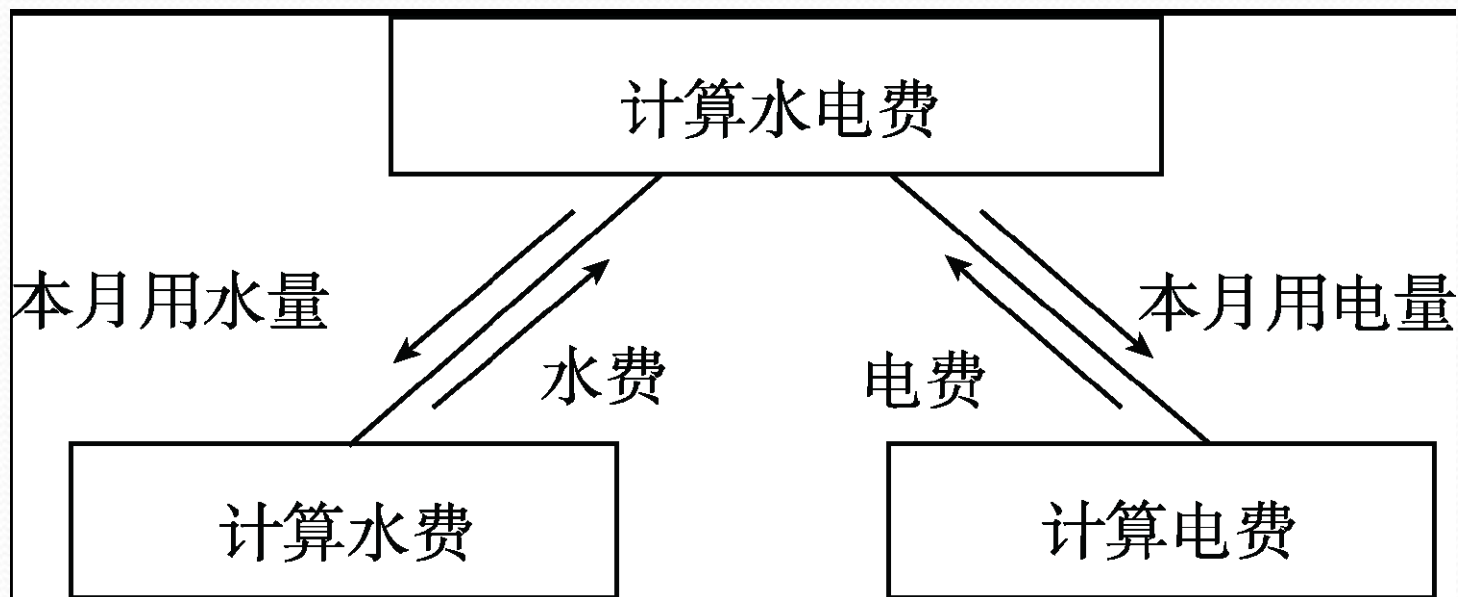
5-标记耦合

- 如果一组模块通过参数表传递记录信息，就是标记耦合。
- 事实上，这组模块共享了某一数据结构的子结构，而不是简单变量。



6-数据耦合

- 如果一个模块访问另一个模块时，彼此之间是通过简单数据参数(不是控制参数、公共数据结构或外部变量)来交换输入、输出信息的，则称这种耦合为数据耦合。

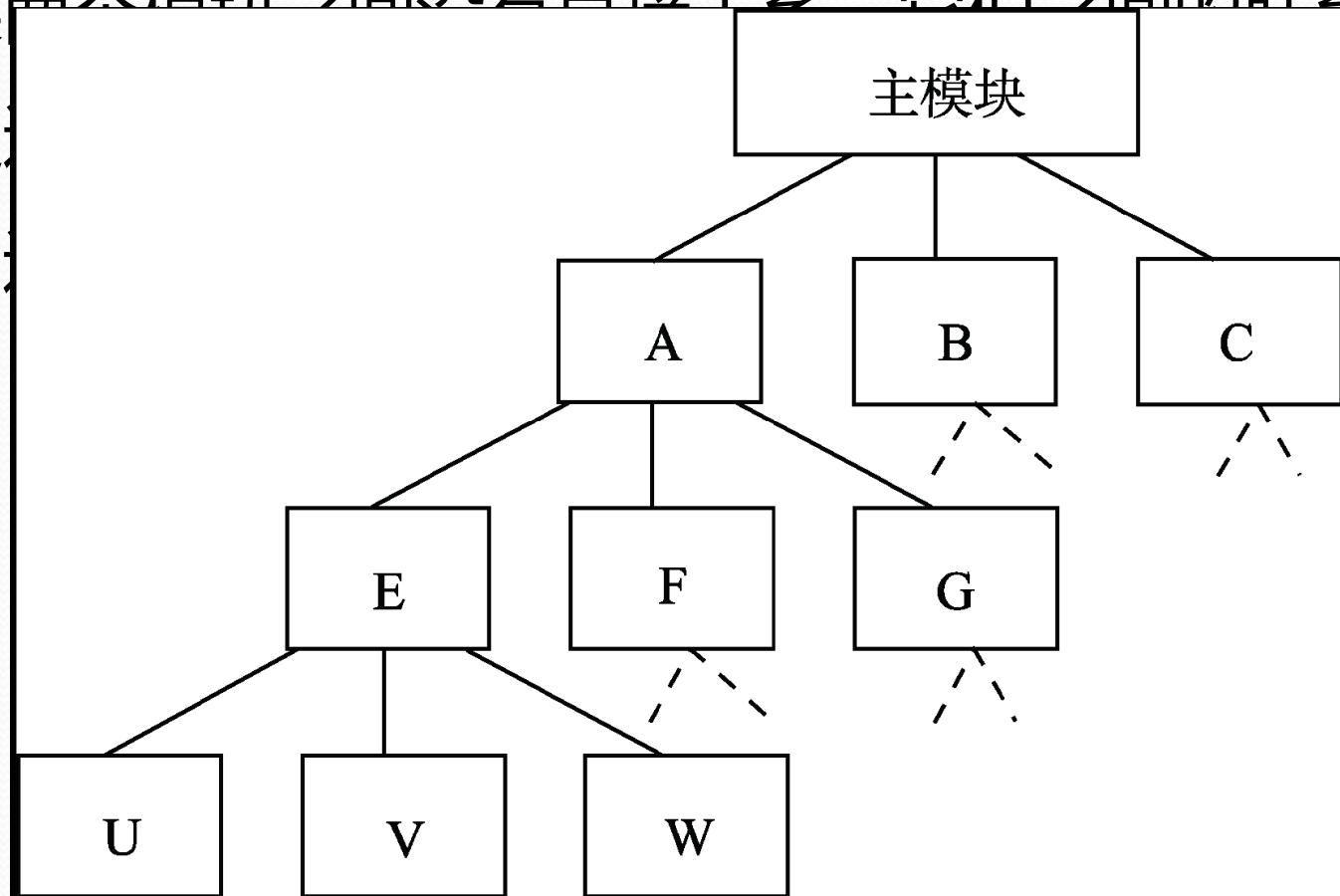


7-非直接耦合

- 如果两个模块之间没有直接关系，它们之间的联系完全是通过主模块的控制和调用来实现的，这就是非直接耦合，这种耦合的模块独立性最强。

7-非直接耦合

- 如果两个模块之间没有直接关系，它们之间的联系完全是通过公共模块实现的，这种耦合称为间接耦合。



模块独立性的意义

- ◆ 在设计软件时应追求尽可能高内聚低耦合的系统
 - 将对这类系统中任一模块的设计、测试和维护相对独立
 - 由于模块间联系较少，错误在模块间传播的可能性也随之变小
 - 模块间的耦合程度直接影响系统的可理解性、可测试性、可靠性和可维护性

2 软件设计的基本原则

◆ 2.1. 模块化

◆ 2.2. 抽象

◆ 2.3. 逐步求精

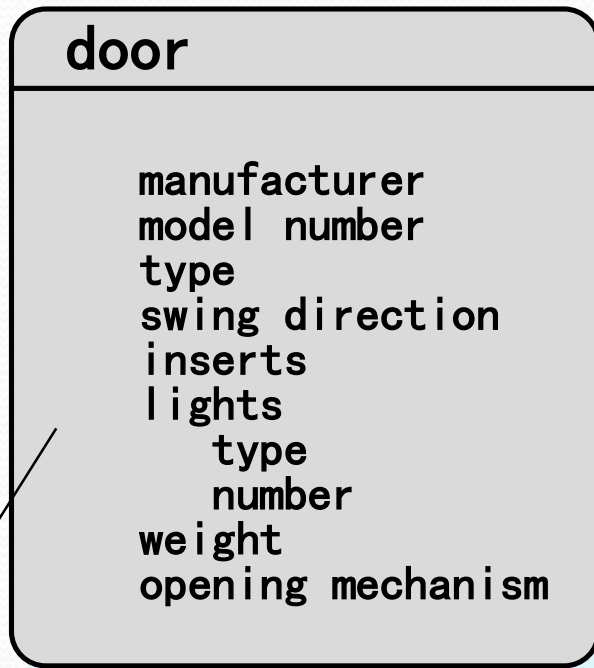
◆ 2.4. 信息隐蔽

2.2 抽象

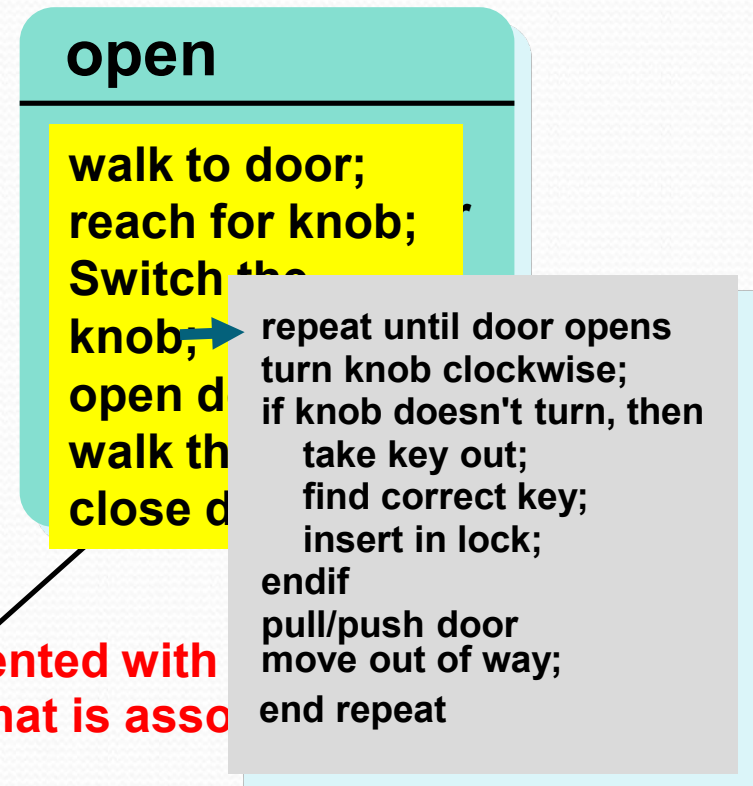
- 2.抽象 Abstraction:
 - 忽略细节，分层理解问题，自顶向下层层细化，包括对过程、数据和控制的抽象。
 - ① 过程抽象：一个命名的指令序列，具有特定和有限的功能。
 - ② 数据抽象：命名的数据集合，描述一个数据对象。
 - ③ 控制抽象：隐含了不必说明的内部细节的程序控制机制。
 - 抽象是实现模块化的手段之一。

2.3求精

- 自顶向下的设计策略。



implemented as a data structure object that is associated with



implemented with

```
repeat until door opens
  turn knob clockwise;
  if knob doesn't turn, then
    take key out;
    find correct key;
    insert in lock;
  endif
  pull/push door
  move out of way;
end repeat
```

2.3求精

- 设计的求精过程与需求分析的划分类似，只是考虑的细节层次不同。
- 求精实际是一个详细描述的过程。
- 抽象与求精是互补的概念。

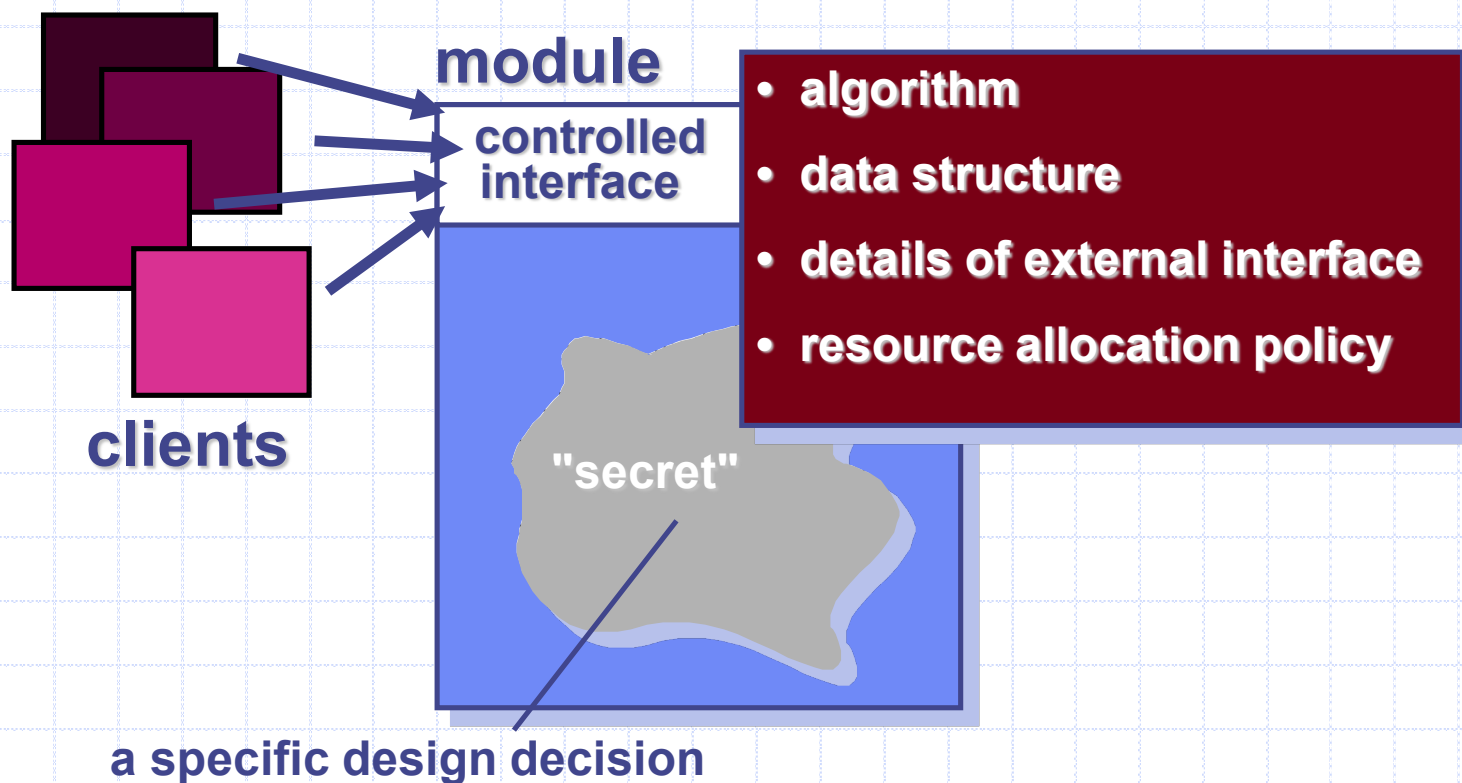
2 软件设计的基本原则

- 2.1. 模块化
- 2.2. 抽象
- 2.3. 逐步求精
- 2.4. 信息隐蔽

2.4 信息隐藏

- 信息隐藏：在设计和确定模块时，使得一个模块内包含的信息（过程和数据）对于不需要这些信息的模块来说，是不可访问的。
- 每个客户只能通过接口来了解该模块，而所有的实现都隐藏起来。
- 隐藏就是有效的模块化可以通过定义一组独立模块来实现。

信息隐藏



“信息隐藏”，更准确地描述应是“**细节隐藏**”，因为隐藏的不是信息，而是实现的细节。

3 结构化软件设计任务

结构化设计过程

结构化分析的结果



概要设计



详细设计

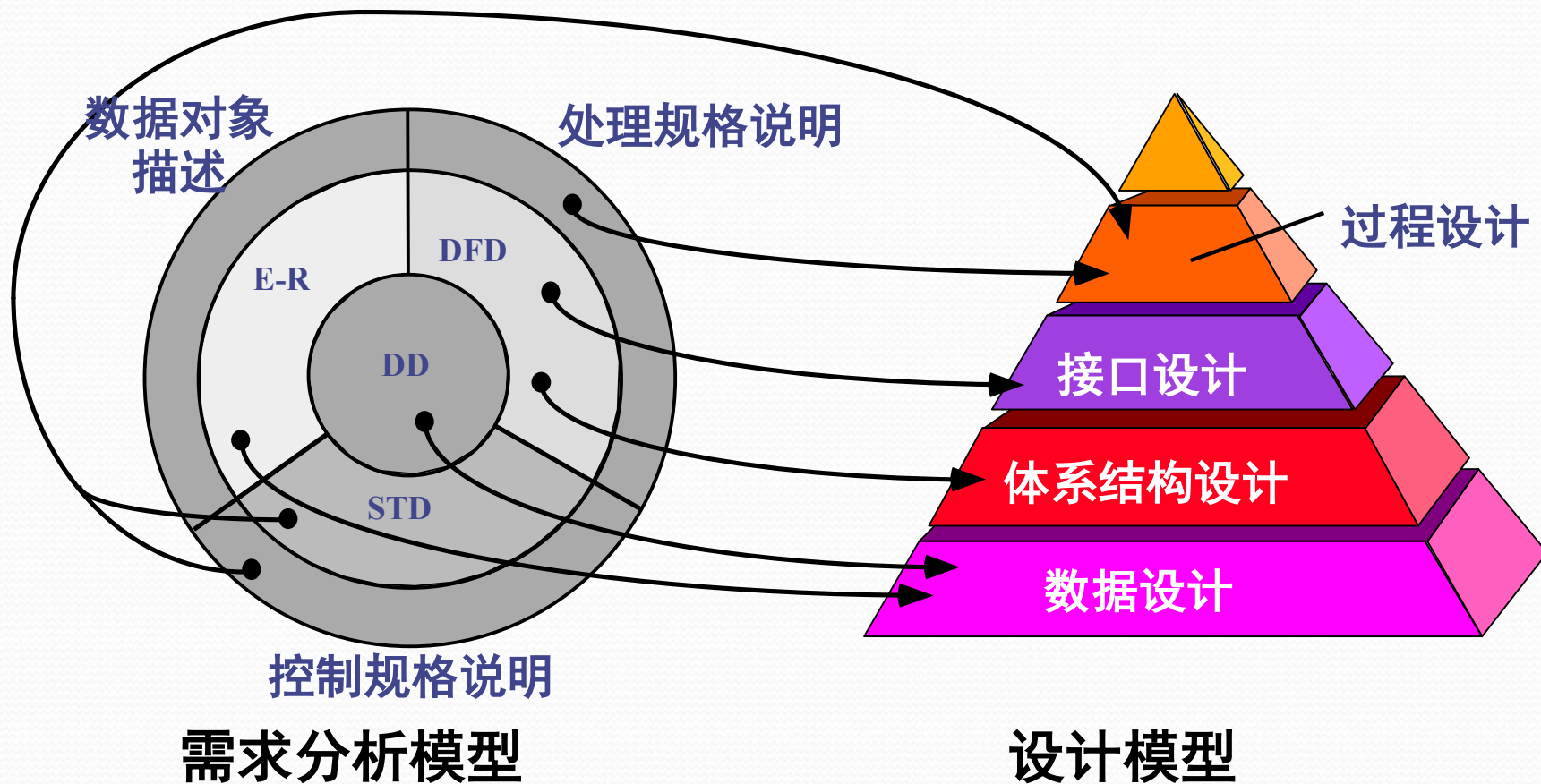


设计复审



软件设计文档

翻译需求模型到软件设计



结构化设计的主要内容

□概要设计

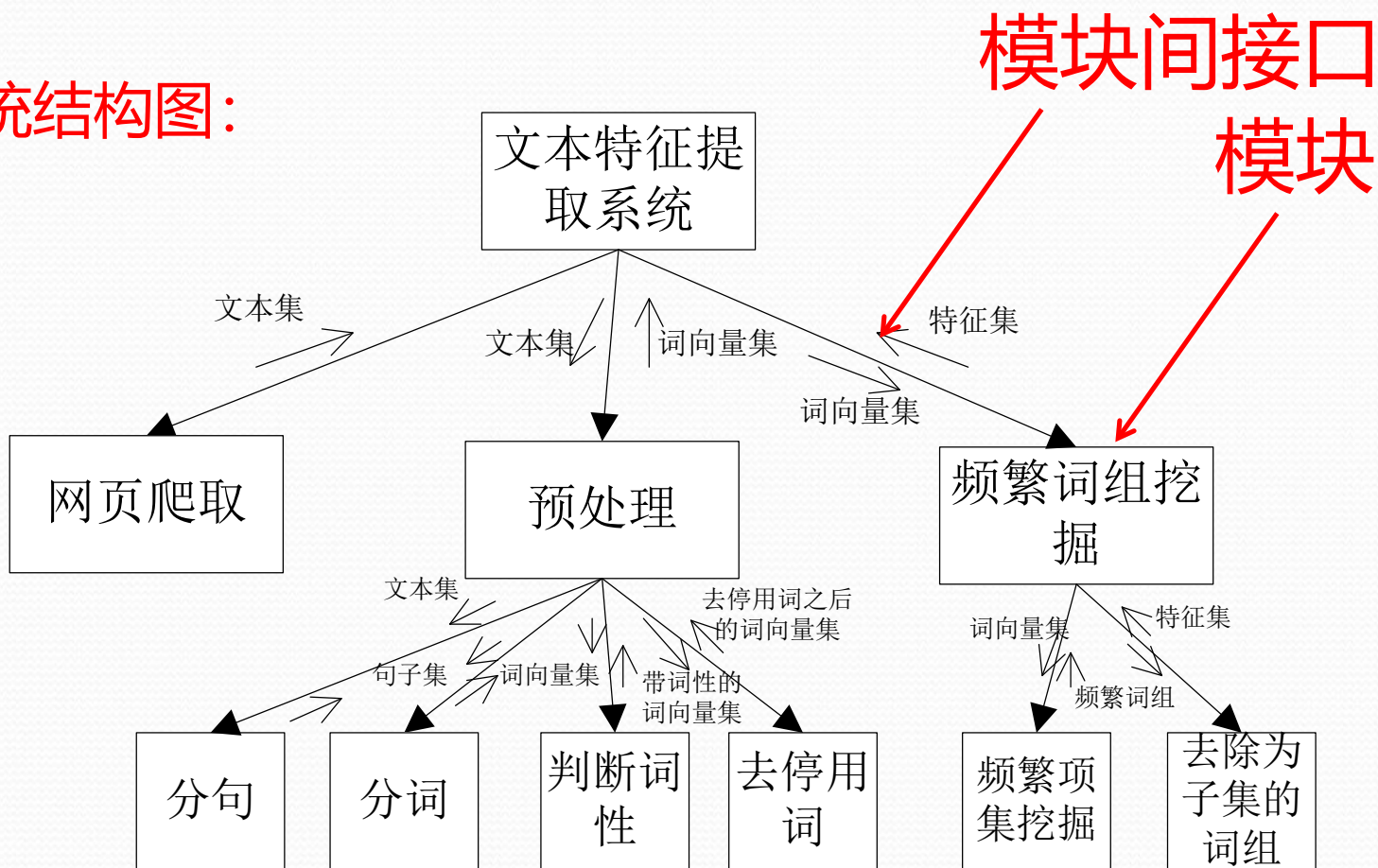
- ✓ 设计软件系统的体系结构（也即软件系统的模块结构）与接口
- ✓ 主要任务是将数据流图映射为软件系统结构图

□详细设计：

- ✓ 对概要设计的每一个模块进行数据设计和过程设计

结构化概要设计：系统结构图

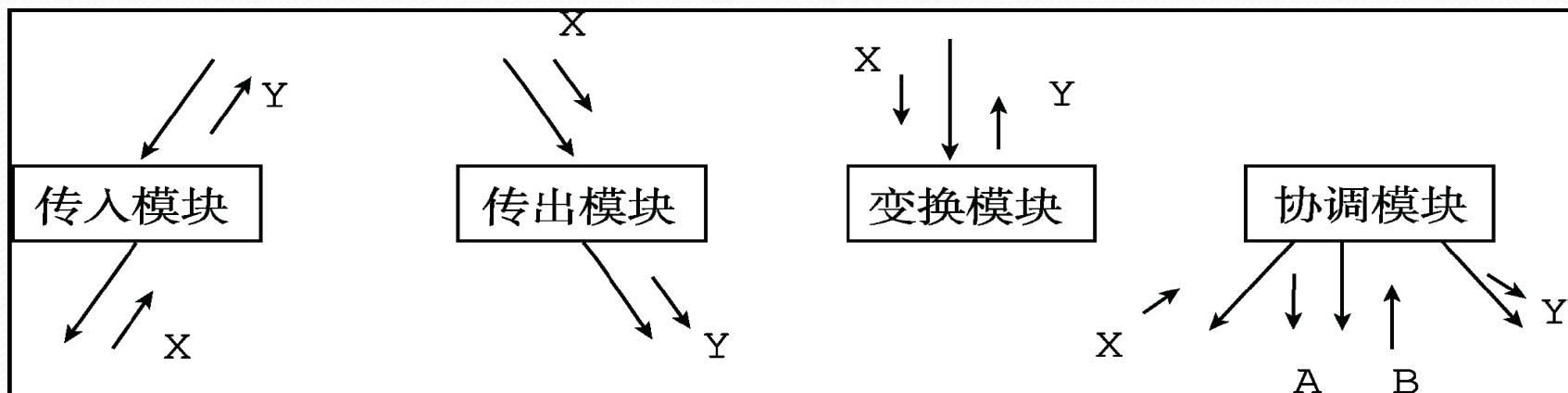
系统结构图：



系统结构图：表现形式

□用矩形框来表示模块

□用带箭头的连线表示模块间的调用关系

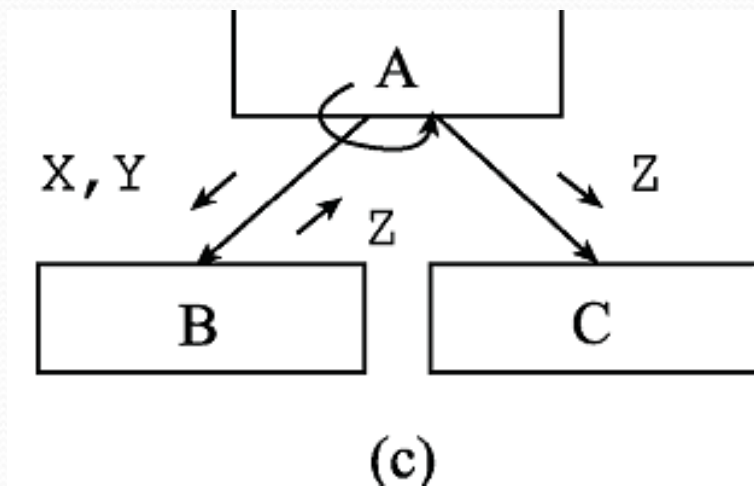
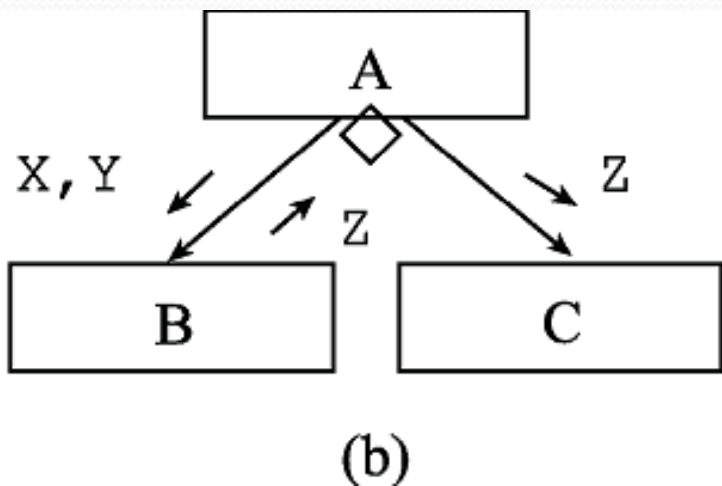
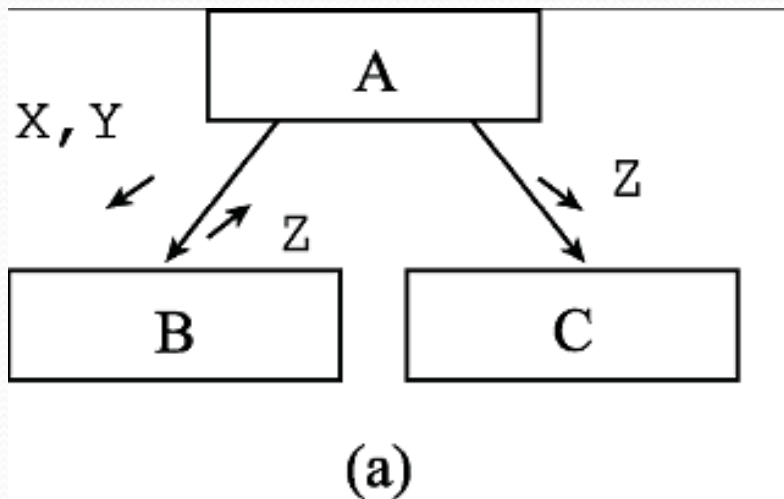


系统结构图：模块间调用关系

□ 简单调用

□ 选择调用

□ 循环调用



两种典型的数据流及其设计

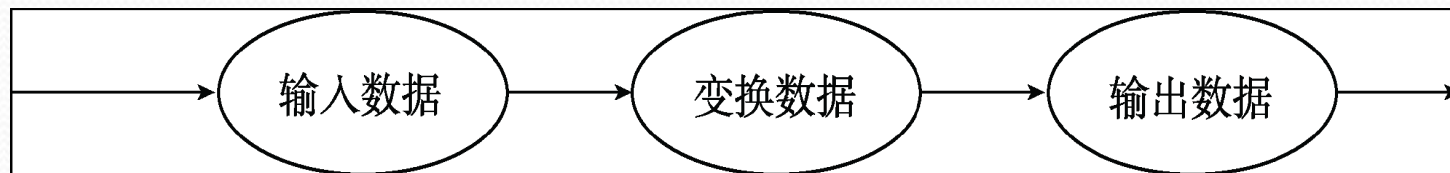
- 变换型数据流图

- 事务型数据流图

两种数据流类型—变换型

□**特征：**信息沿输入通路进入系统，同时由外部形式变换成内部形式，进入系统的信息通过变换中心，经加工处理以后再沿输出通路变换成外部形式离开软件系统。

□**组成：**由输入、变换中心和输出三部分组成



取得数据

变换中心

给出数据

输入
信息

正确
信息

结果

数据

格式
检查

处理

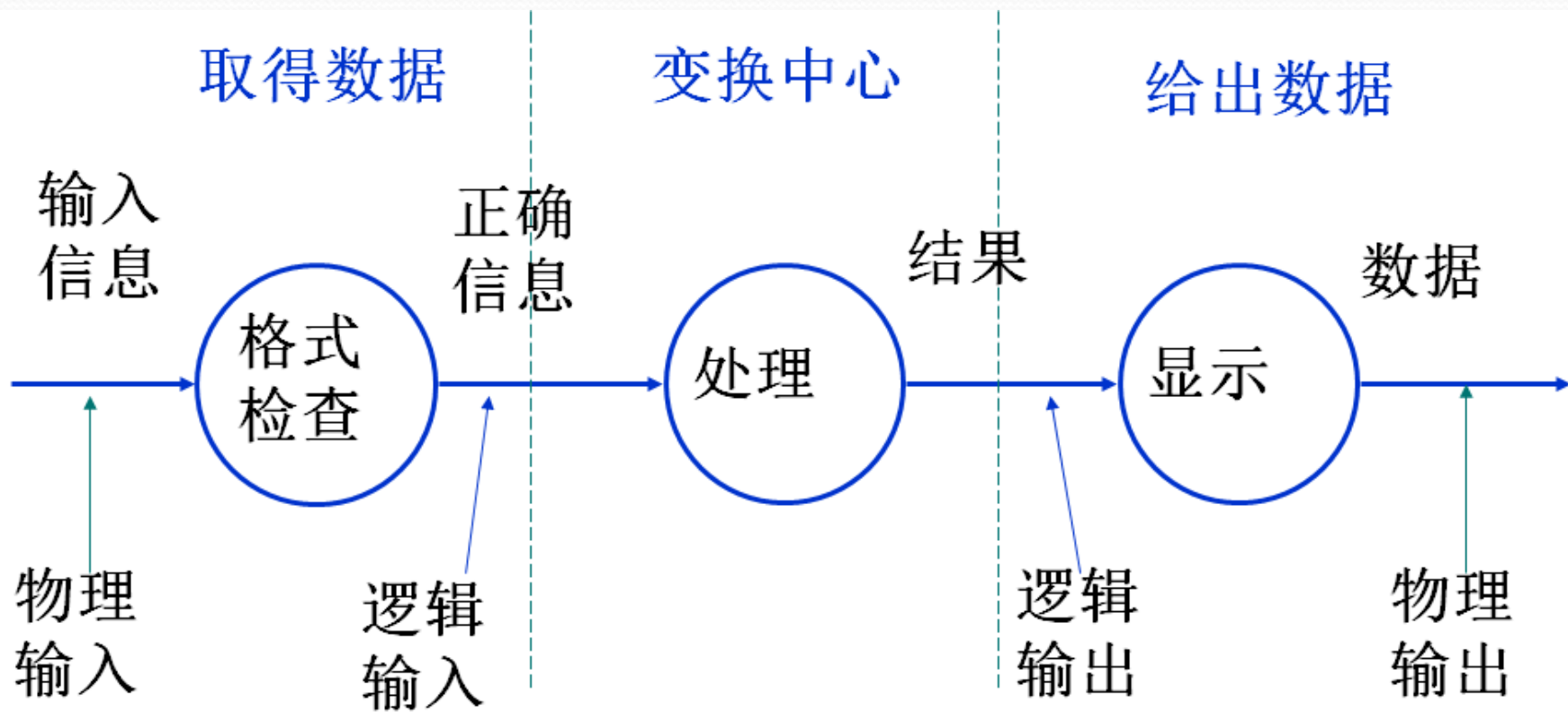
显示

物理
输入

逻辑
输入

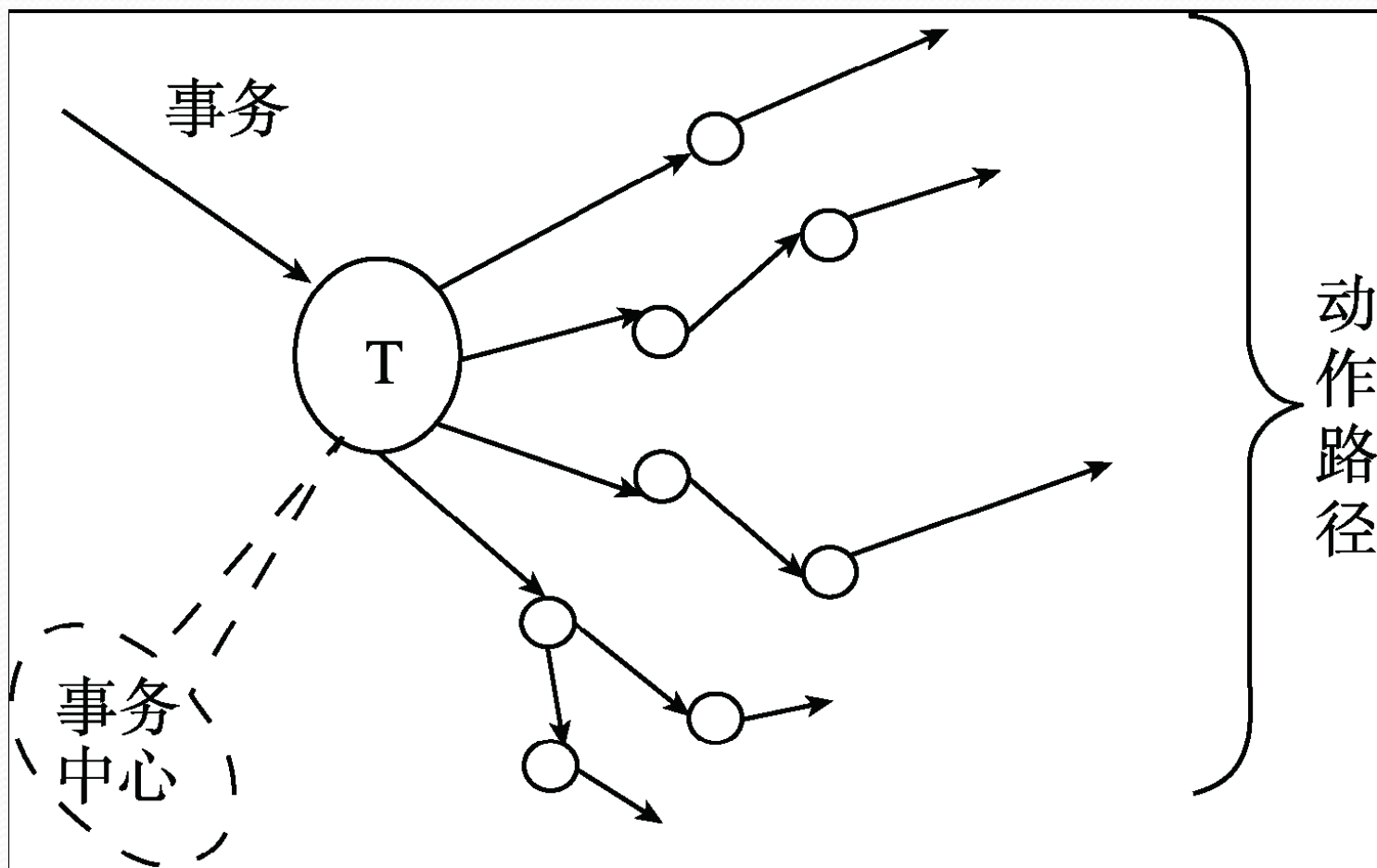
逻辑
输出

物理
输出



两种数据流类型—事务型

- **特征：**数据沿输入通路到达一个加工T，这个加工根据输入数据的类型在若干个动作序列中选出一个来执行。
- 加工T 称为**事务中心**，它完成以下任务：
 - (1) 接收输入数据(输入数据又称为事务)。
 - (2) 分析每个事务以确定它的类型。
 - (3) 根据事务类型选取一条动作路径。



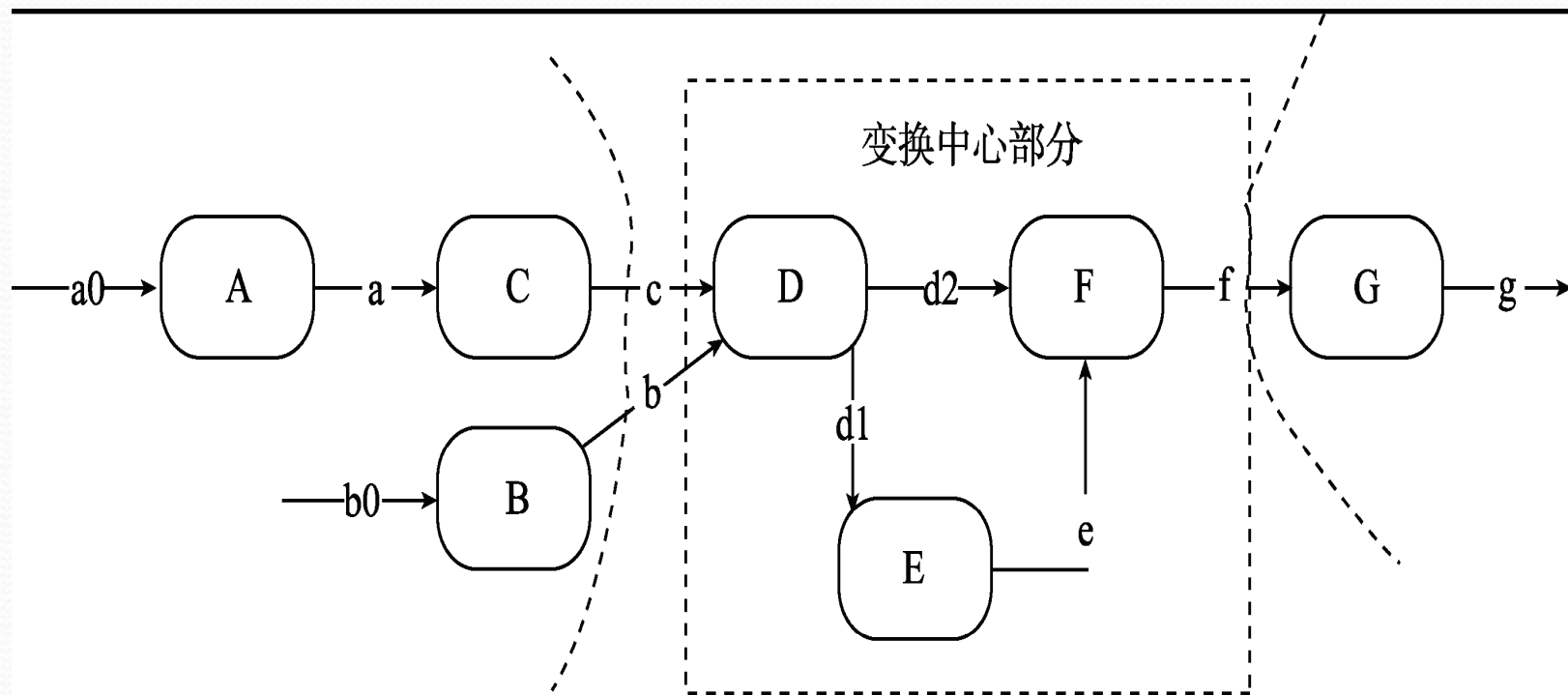
变换型数据流对应的设计： 变化分析

□变换分析是一系列设计步骤的总称，经过这些步骤把具有变换流特点的数据流图映射成为一个预定义的程序结构模板。

□变换分析的步骤：

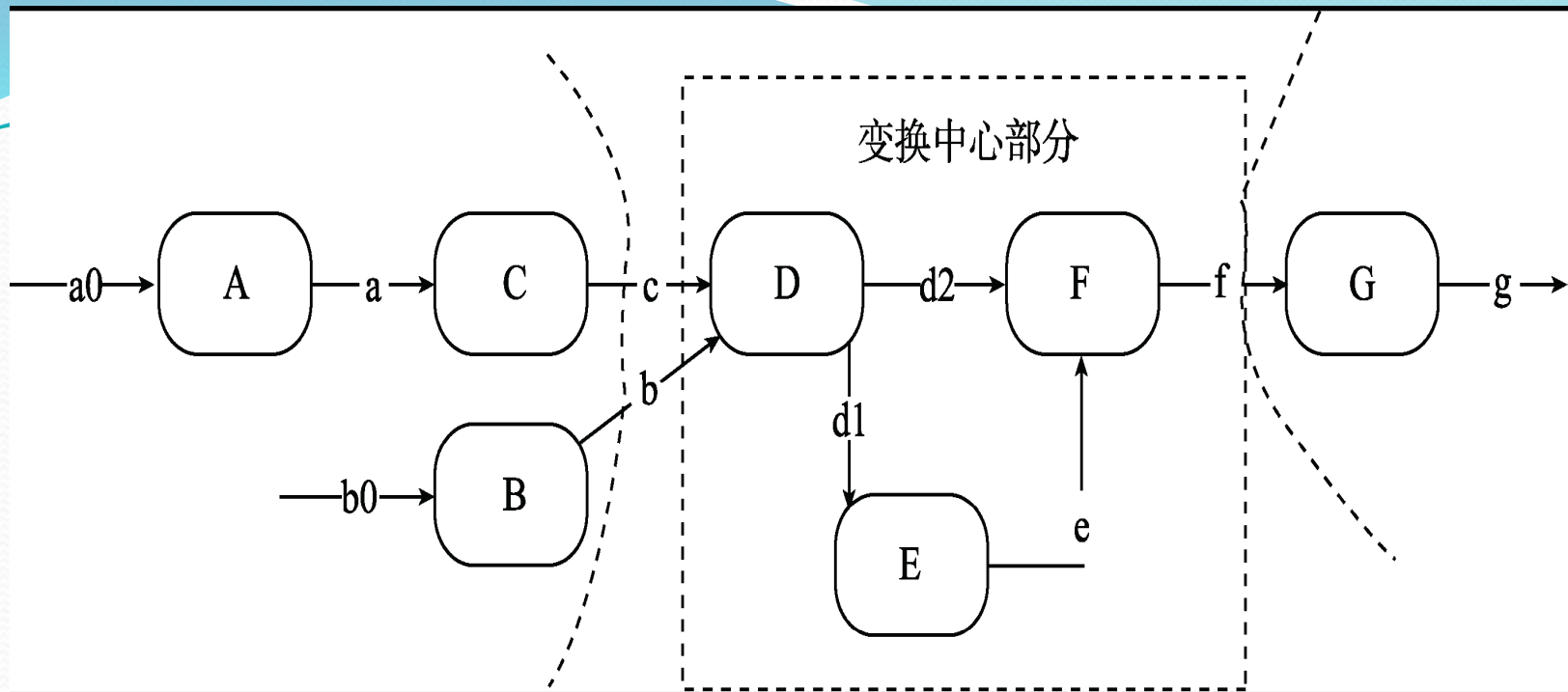
- ✓ 1. 对DFD 图的分析 and 划分
- ✓ 2. 进行第一级分解，设计顶层和第一层模块
- ✓ 3. 第二级分解，设计中、下层模块

一个例子

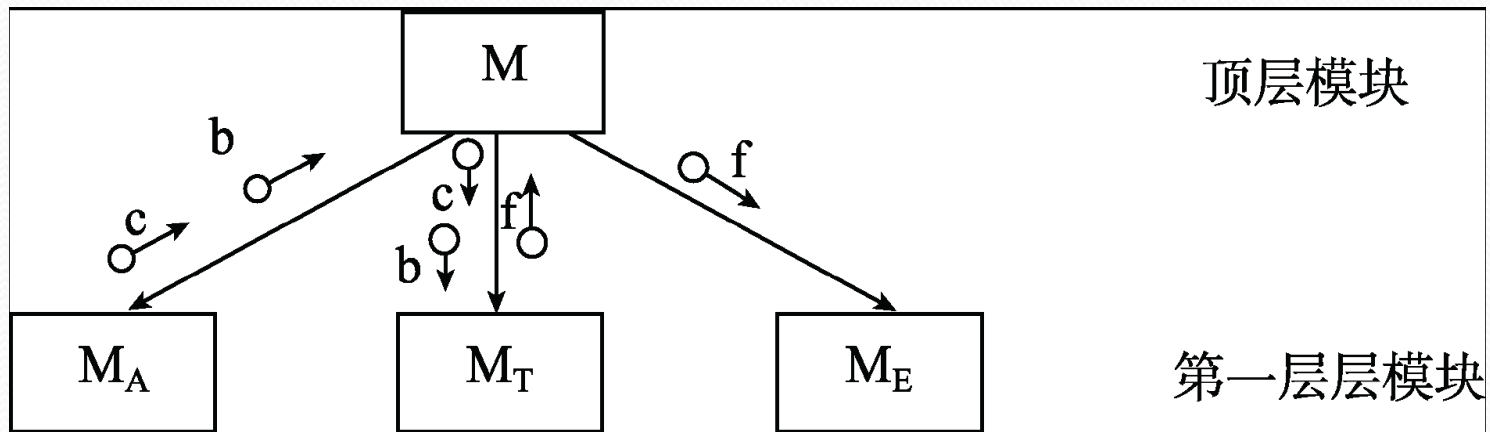


进行第一级分解，设计顶层和第一层模块（1/2）

- 顶层都只有一个用于控制的主模块。
- 第一层一般包括输入、输出和变换3 个模块
 - ✓ **输入模块**：其功能是向主模块提供数据；
 - ✓ **输出模块**：其功能是把主模块提供的数据输出；
 - ✓ **变换模块**：其功能是把逻辑输入变换成逻辑输出。

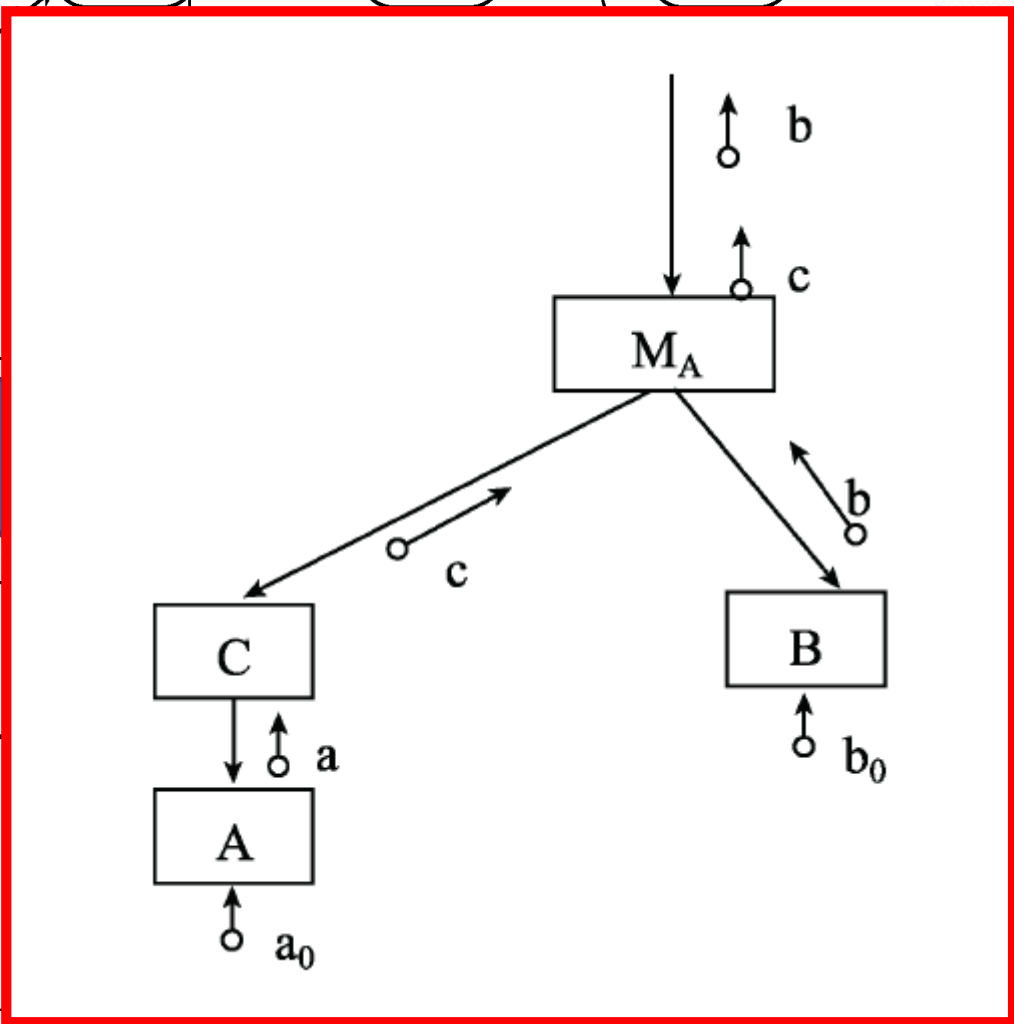
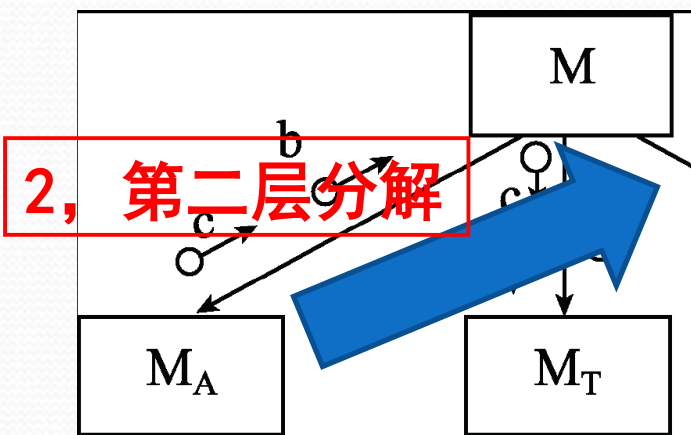
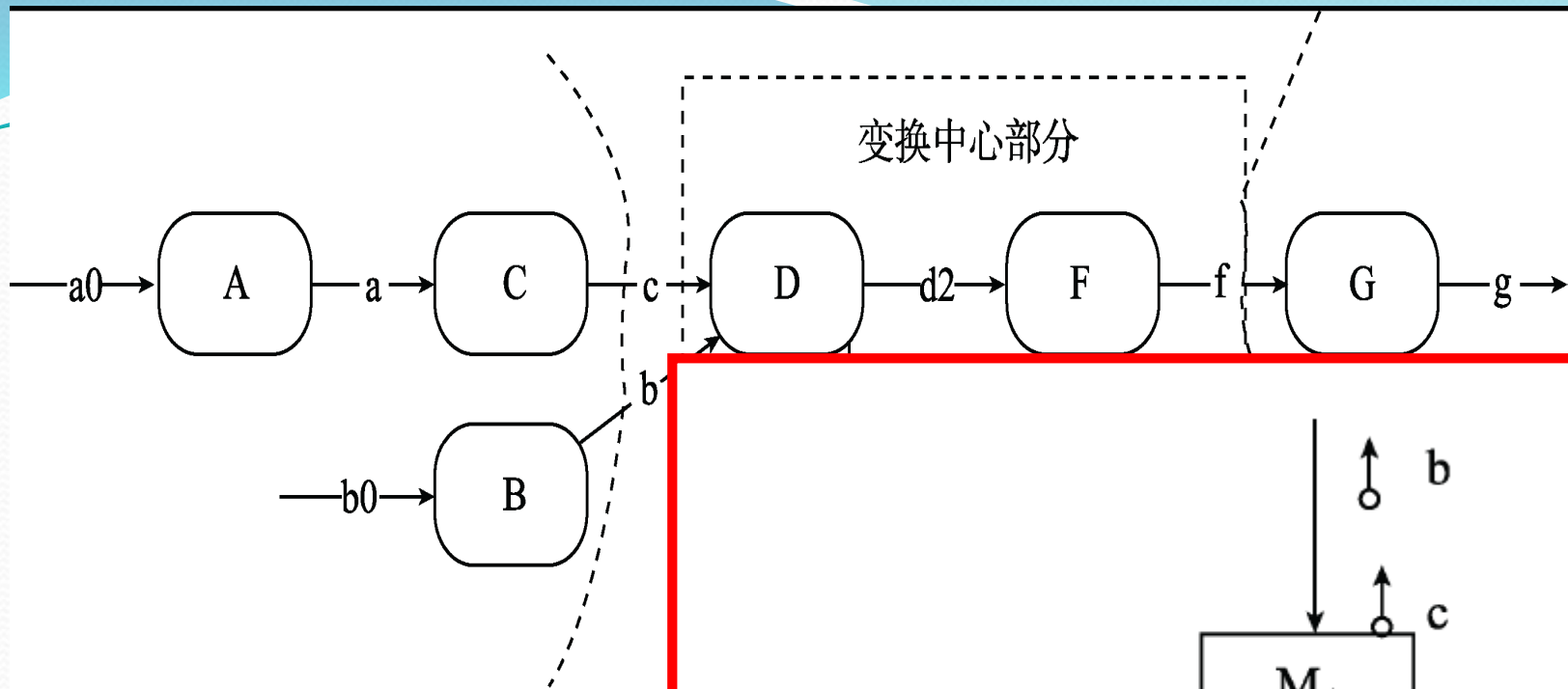


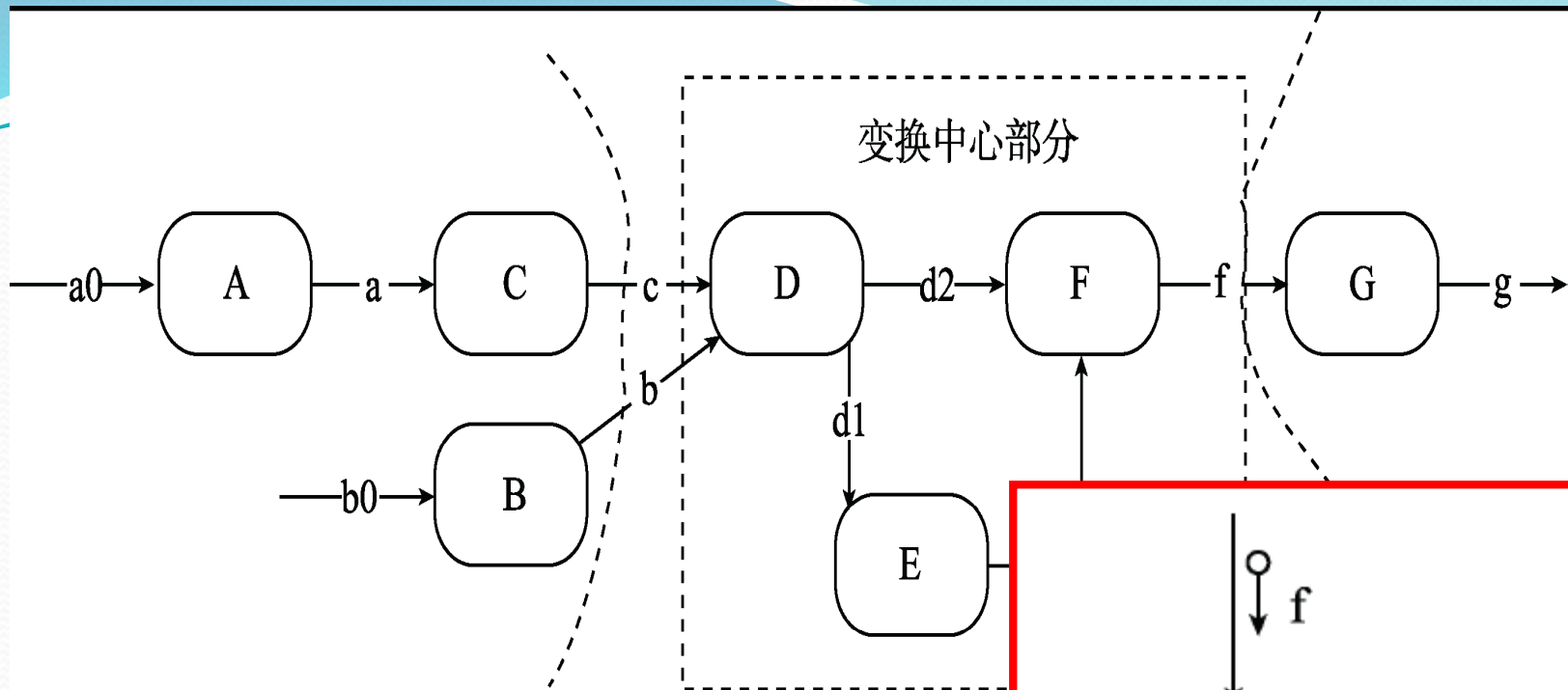
1, 第一层分解, 设计顶层和第一层



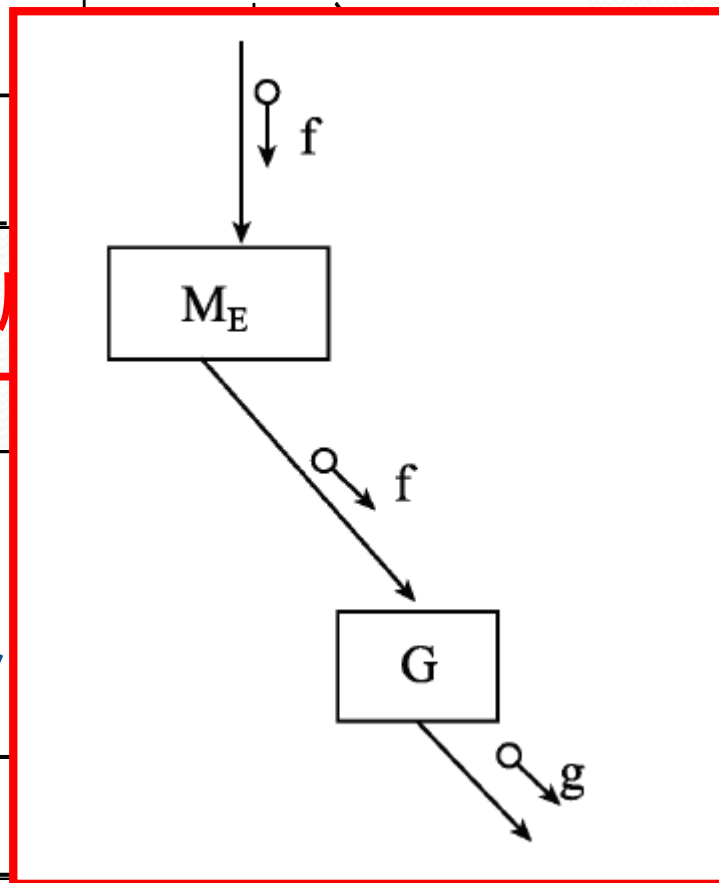
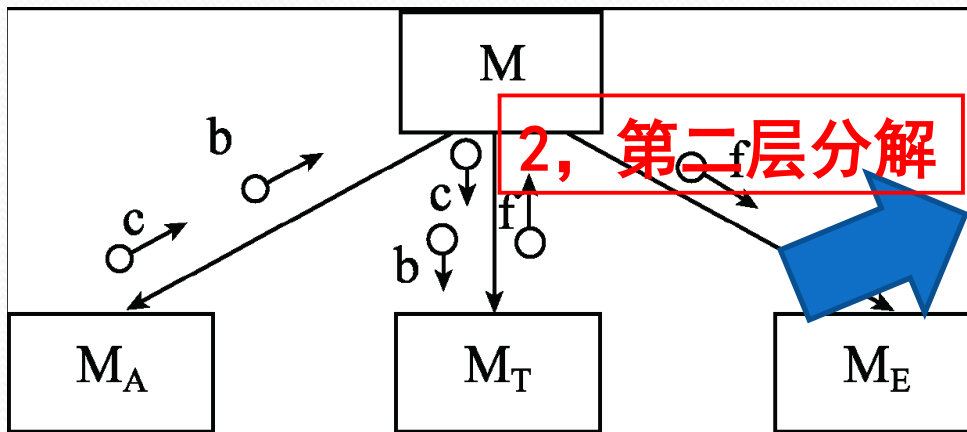
第二级分解，设计中、下层模块（1/2）

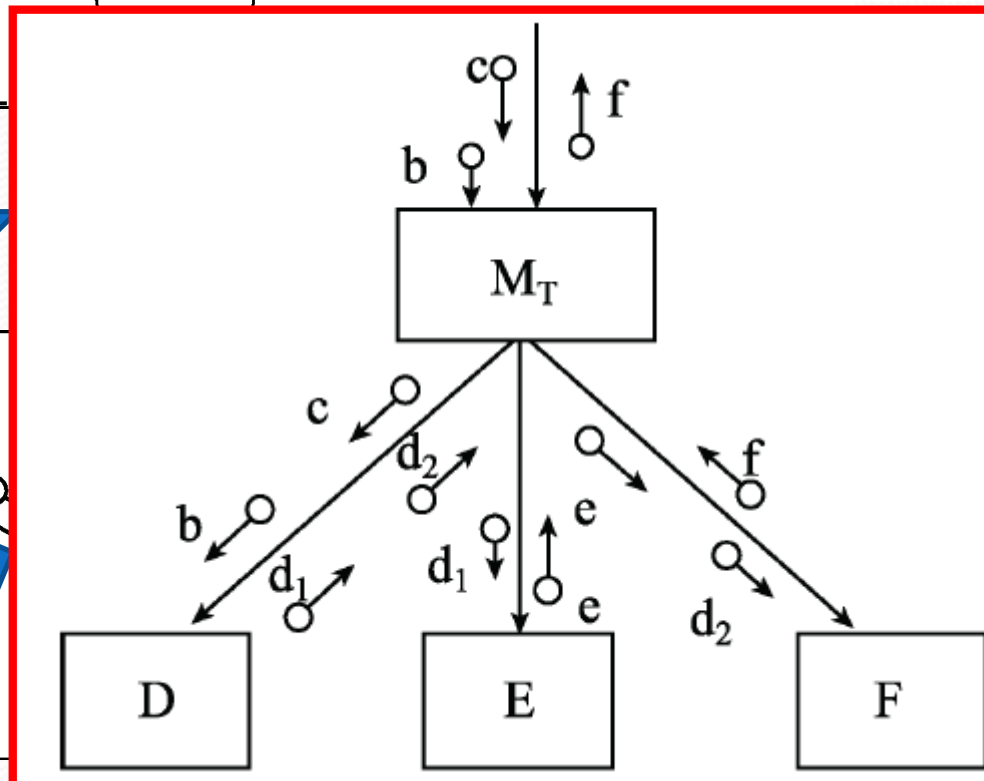
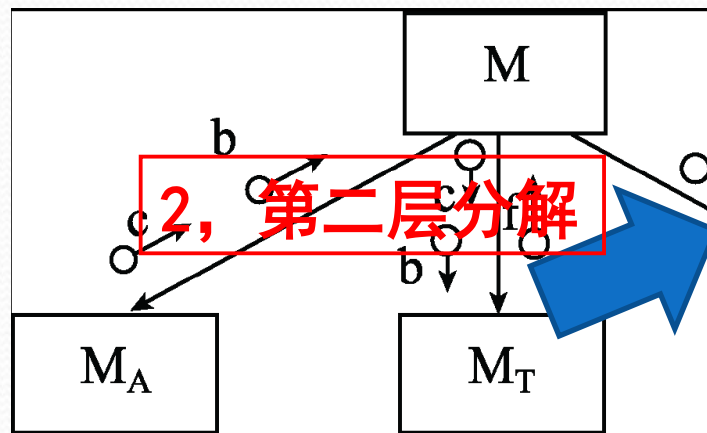
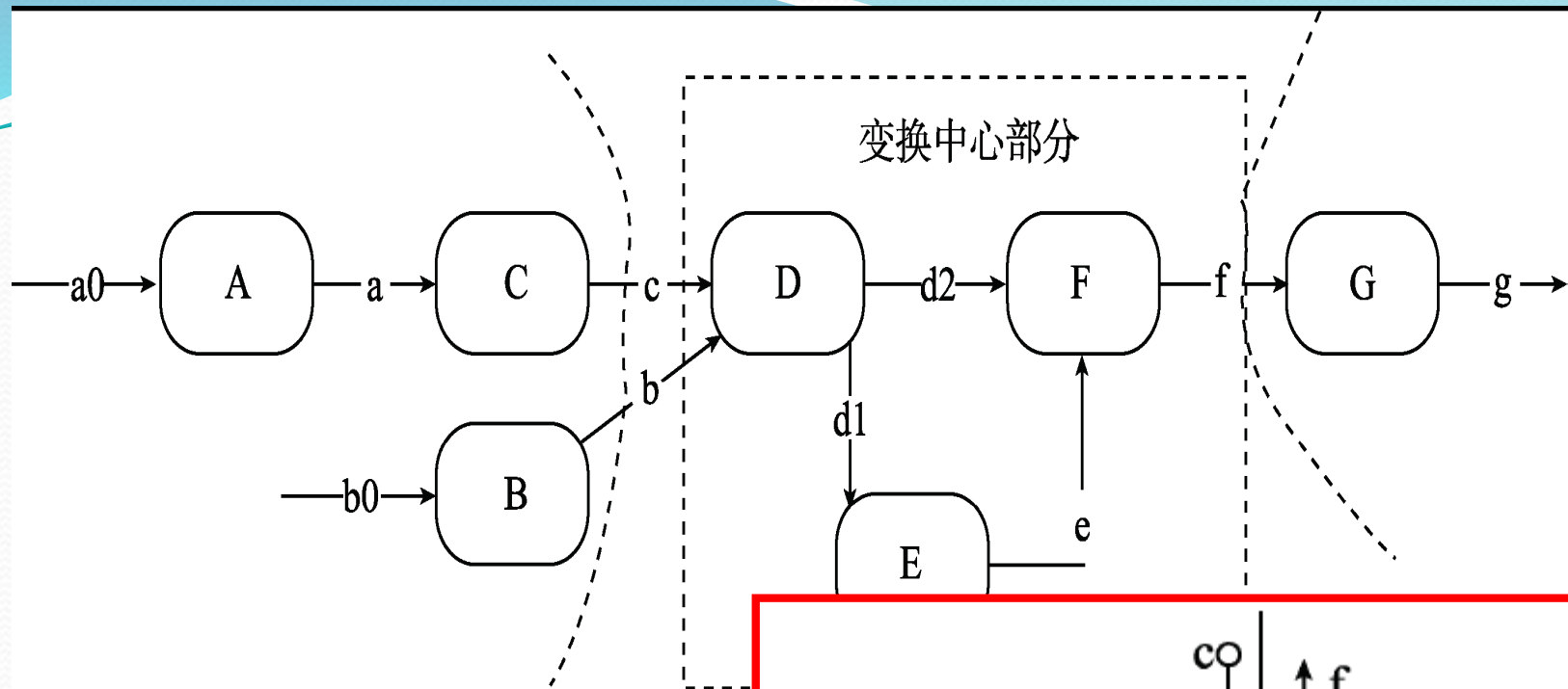
- 第二级分解就是把数据流图中的每个加工映射成软件结构中的一个适当的模块。

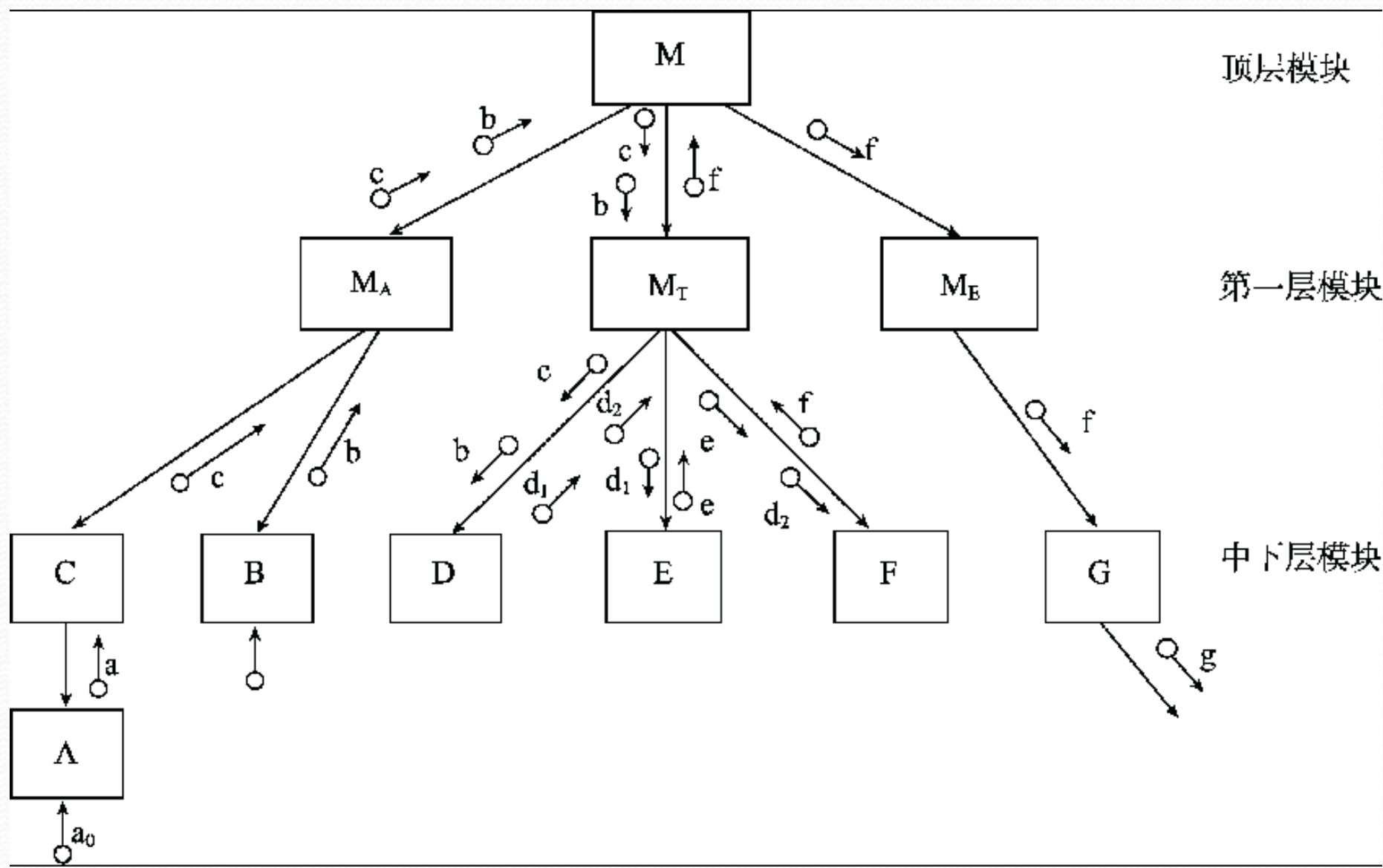




1, 第一层和第一





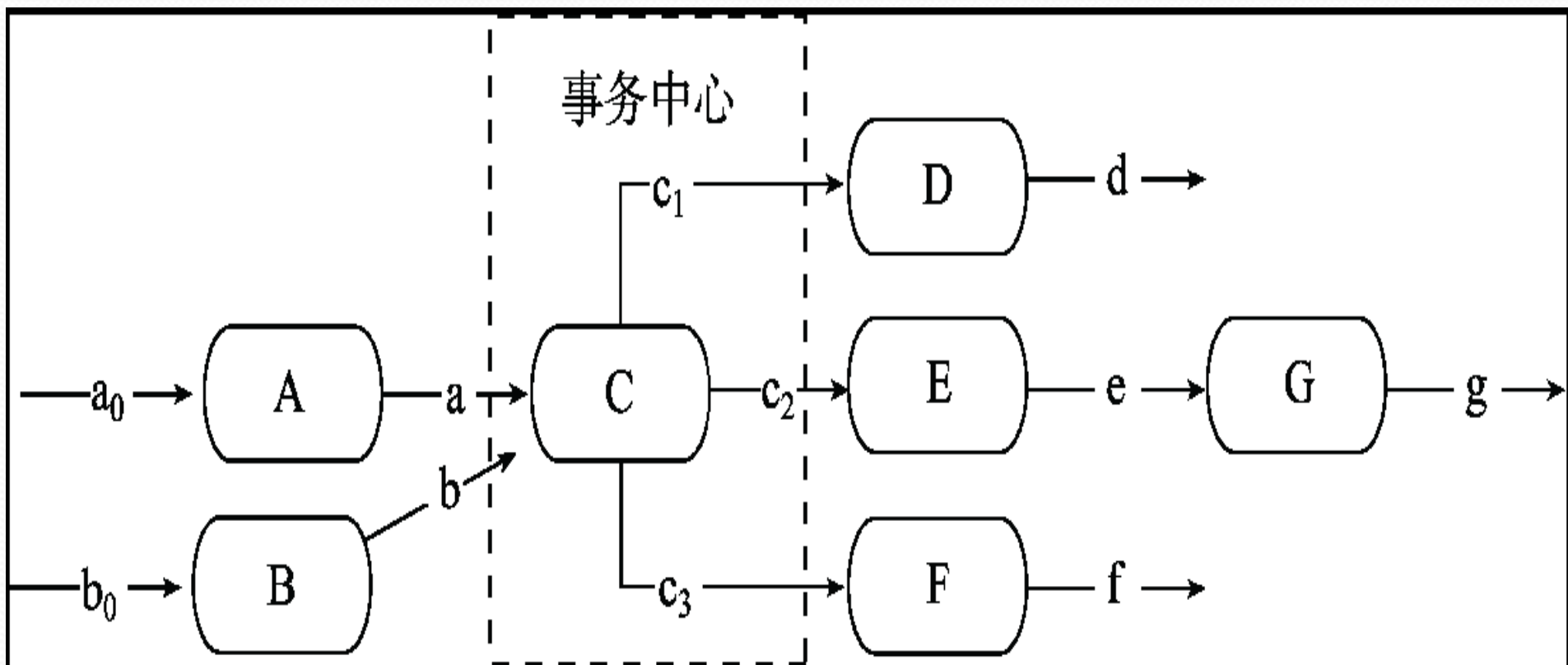


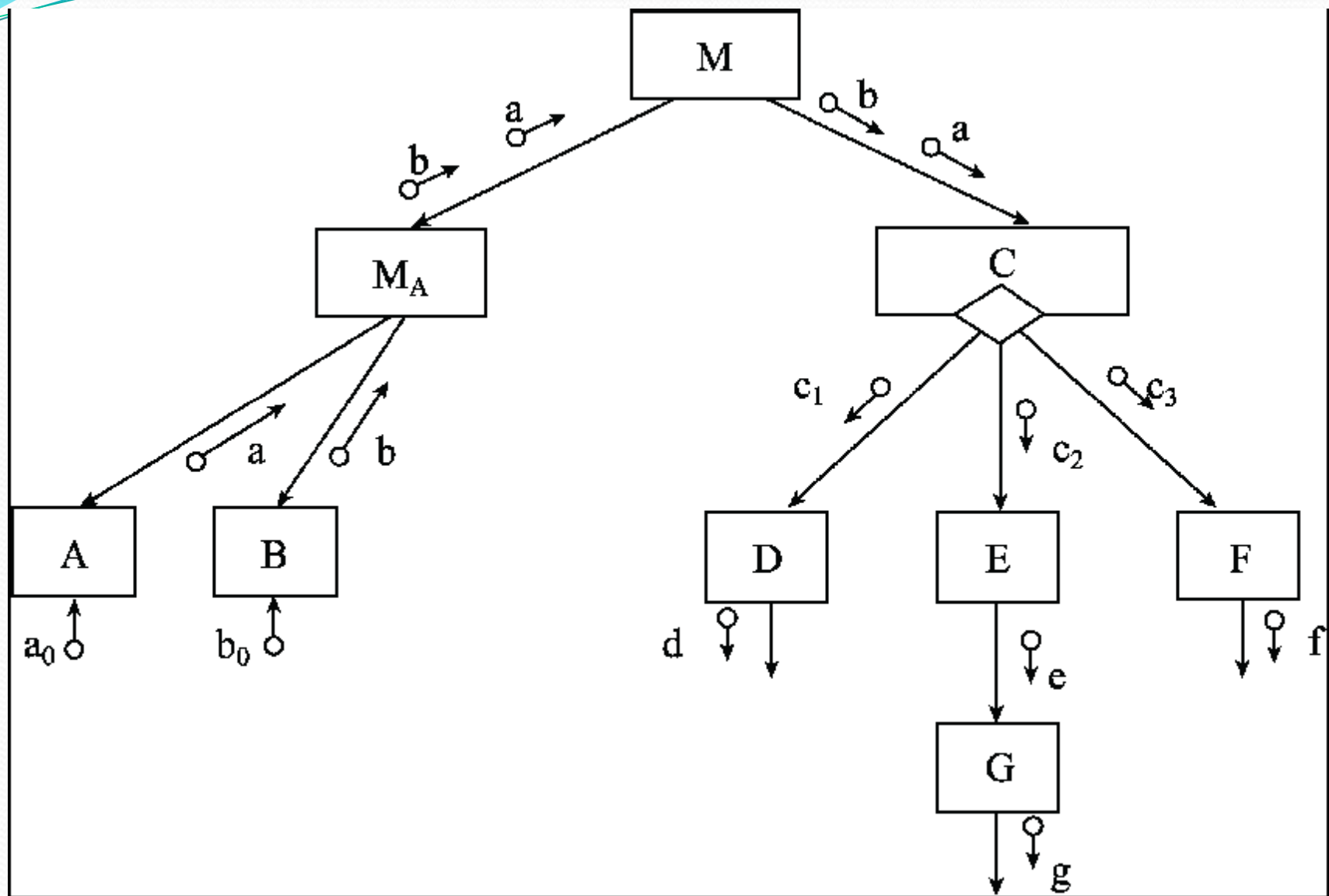
事务型数据流对应的设计：

事务分析

□**事务分析**：由事务型数据流图映射为软件结构的过程。

一个例子





模块分解中的启发式规则

- 1. 改进软件结构以提高模块独立性
- 2. 模块的大小要适当
- 3. 深度、宽度、扇出和扇入都应适当
- 4. 模块的作用域应在控制域之内
- 5. 力争降低模块接口的复杂程度
- 6. 设计单入口单出口模块
- 7. 模块功能应该可预测，避免对模块施加过多限制

模块分解中的启发式规则

- 1. 改进软件结构以提高模块独立性
- 2. 模块的大小要适当

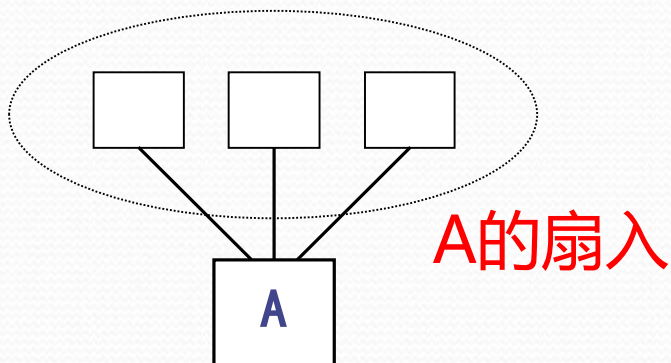
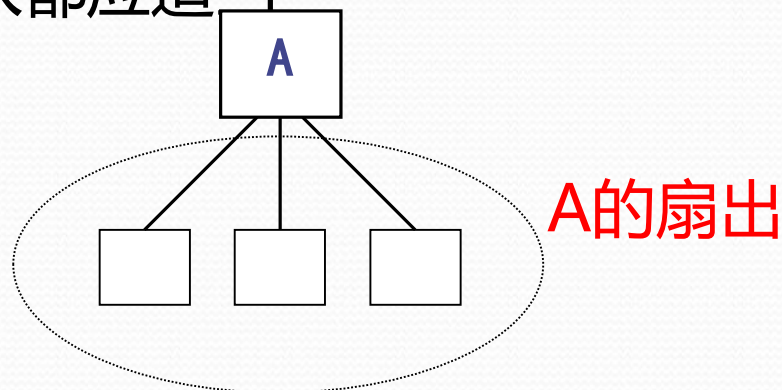
经验表明，一个模块的规模不应过大，通常规定其语句行数为50~100 行，最多不超过500 行。

模块分解中的启发式规则

□ 3. 深度、宽度、扇出和扇入都应适当

扇出 = 一个模块直接调用\控制的模块数。

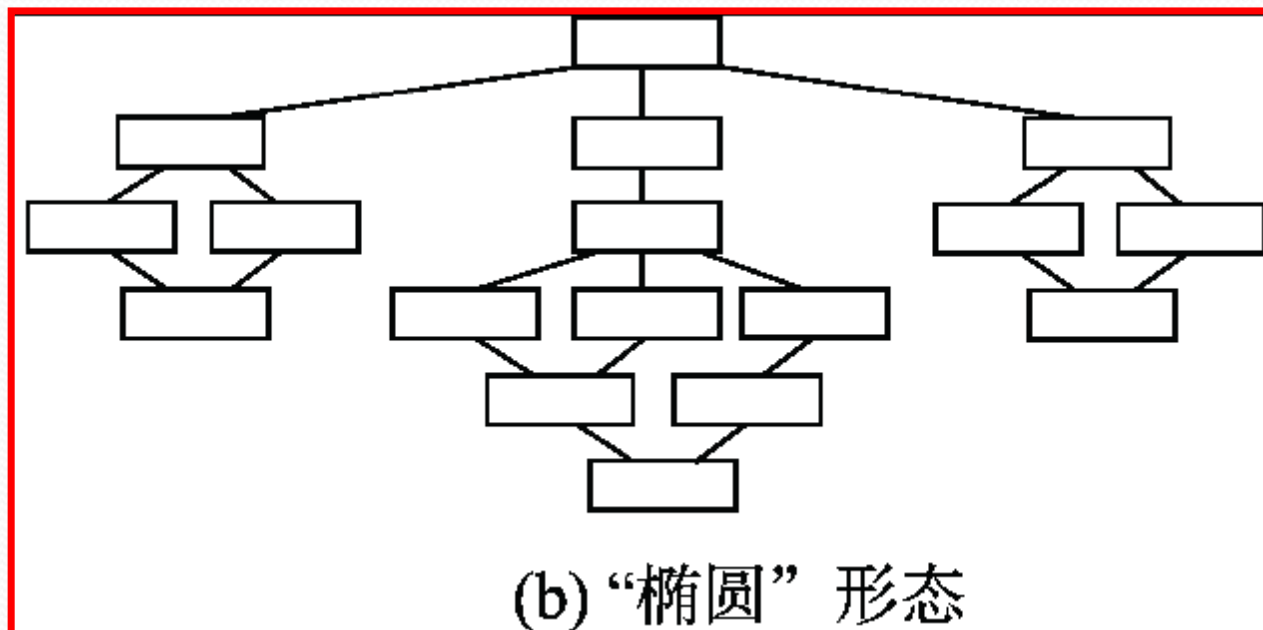
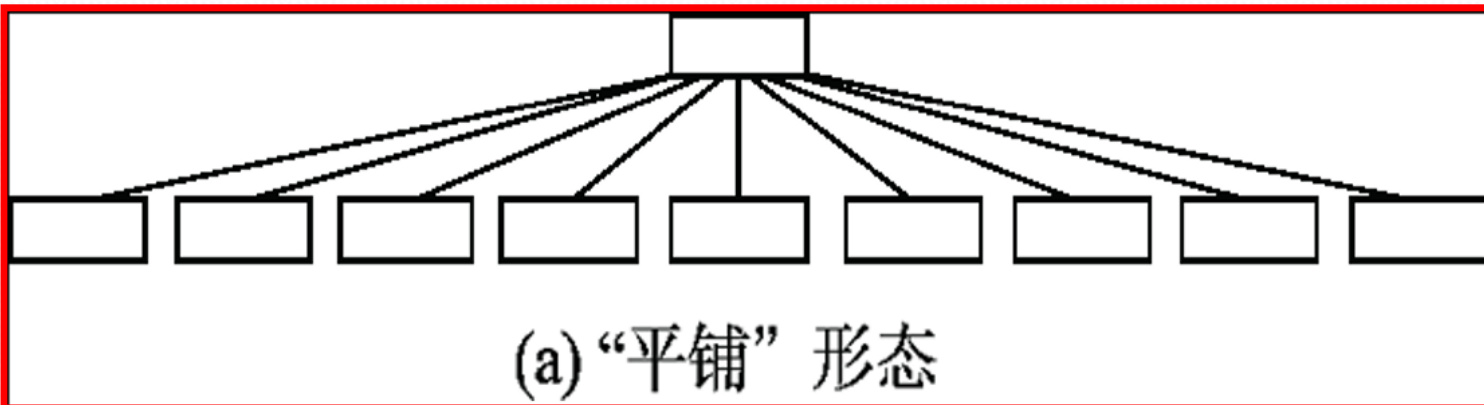
$$3 \leq \text{fan-out} \leq 9$$



扇入 = 直接调用该模块的模块数。在不破坏独立性的前提下, fan-in 大的比较好。

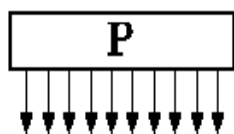
模块分解中的启发式规则

□ 3. 深度、宽度、扇出和扇入都应适当

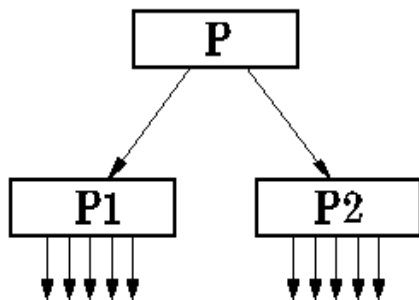


模块分解中的启发式规则

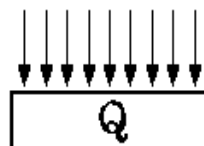
- **尽可能减少高扇出结构，随着深度增大扇入。**
如果一个模块的扇出数过大，就意味着该模块过分复杂，需要协调和控制过多的下属模块。应当适当增加中间层次的控制模块。
- 一般来说，顶层扇出高，中间扇出少，低层高扇入。



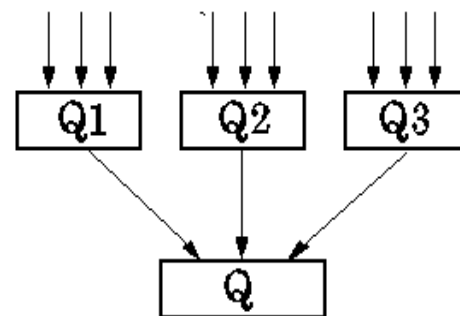
(a)



(b)

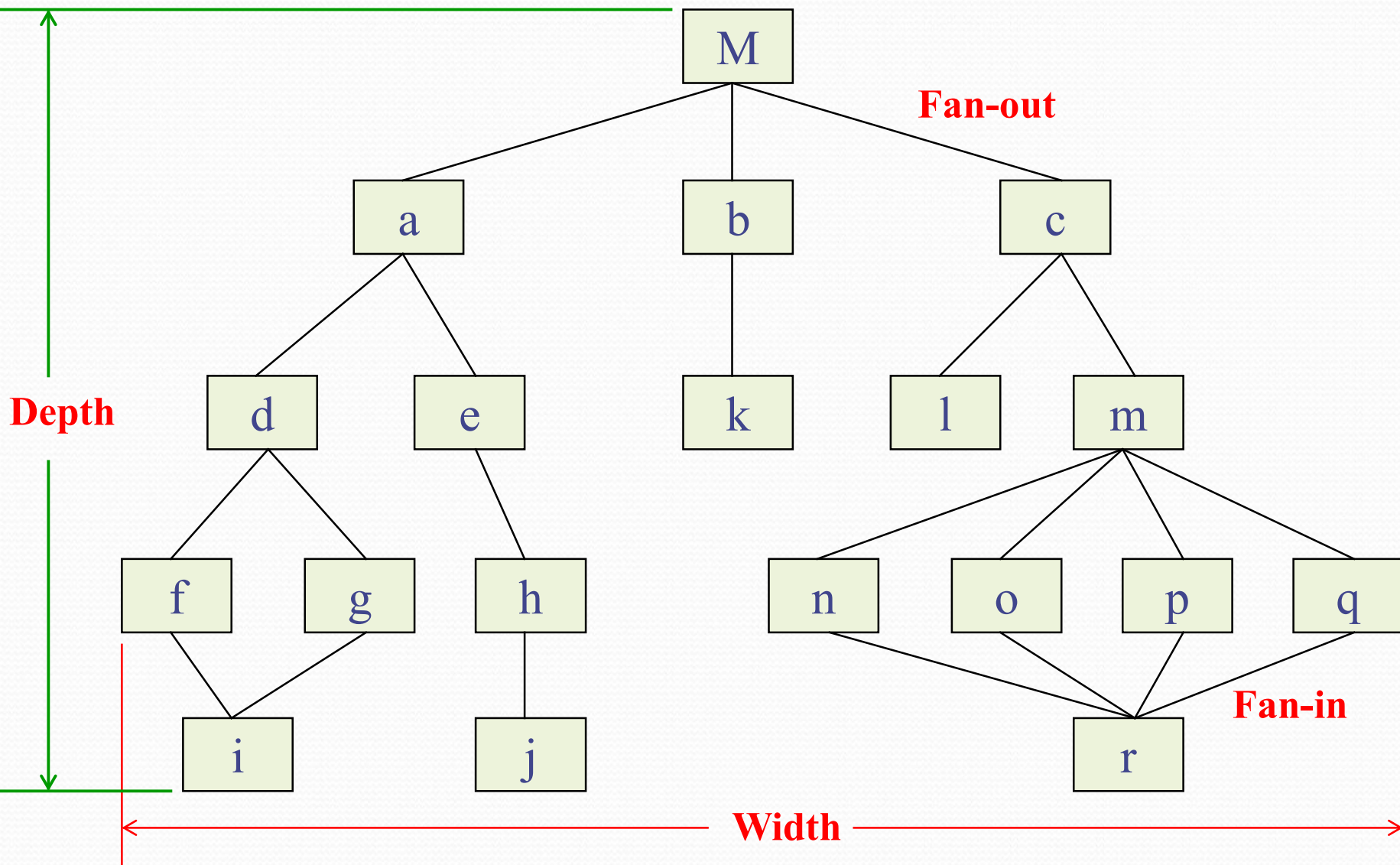


(c)



(d)

控制层次



模块分解中的启发式规则

□ 4. 模块的作用域应在控制域之内

模块的作用域定义为受该模块内一个判定影响的所有模块的集合。

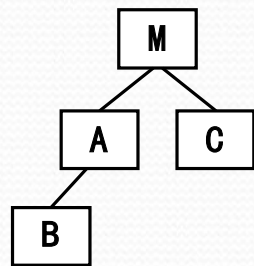
模块的控制域是指这个模块本身以及所有直接或间接从属于它的模块的集合。

模块分解中的启发式规则

4. 模块的作用域应在控制域之内

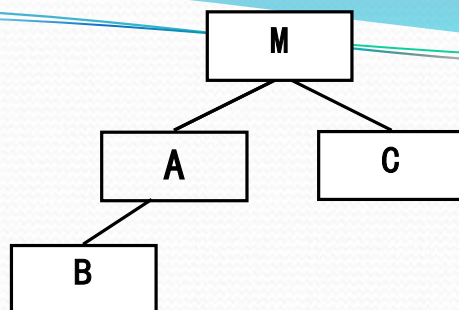
- 作用域是指该模块中一个判断所影响的所有其它模块；
- 控制域指该模块本身以及所有直接或间接从属于它的模块。

◆ 控制域



M的控制域为 {M, A, B, C}

- ### ◆ 作用域
- M中的一个判定所影响的模块。



● 例：

```

A:
  .....
  if .....
  then goto
B1
  .....
  .....
  
```

作用域在控制域内

```

B:
  .....
  .....
B1:
  .....
  .....
  
```

```

A:
  .....
  if .....
  then goto
M1
  .....
  .....
  
```

作用域超出了控制域

```

M:
  .....
  .....
M1: goto C1
  .....
  .....
  
```

上例中A的作用超出了控制域。

改进方法之一，可以把A中的 if 移到M中；

改进方法之二，可以把C移到A下面。

详细设计

□ 详细设计就要对软件结构中的**每一个模块**确定使用的**算法**或块**内数据结构**，并用某种选定的表达工具给出清晰的描述。

□ 详细设计的表达工具

- ✓ 程序流程图

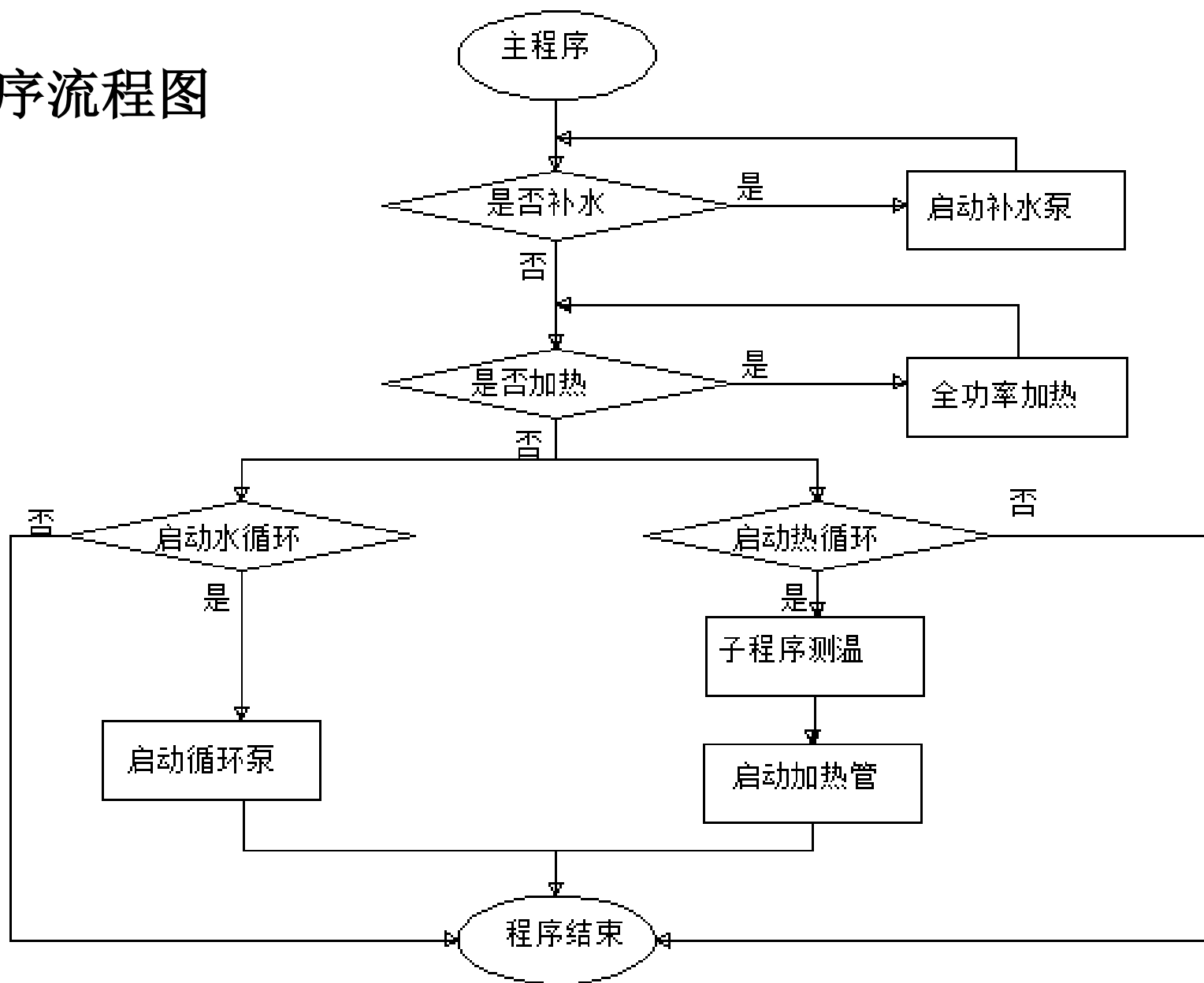
- ✓ 盒图

- ✓ PAD 图

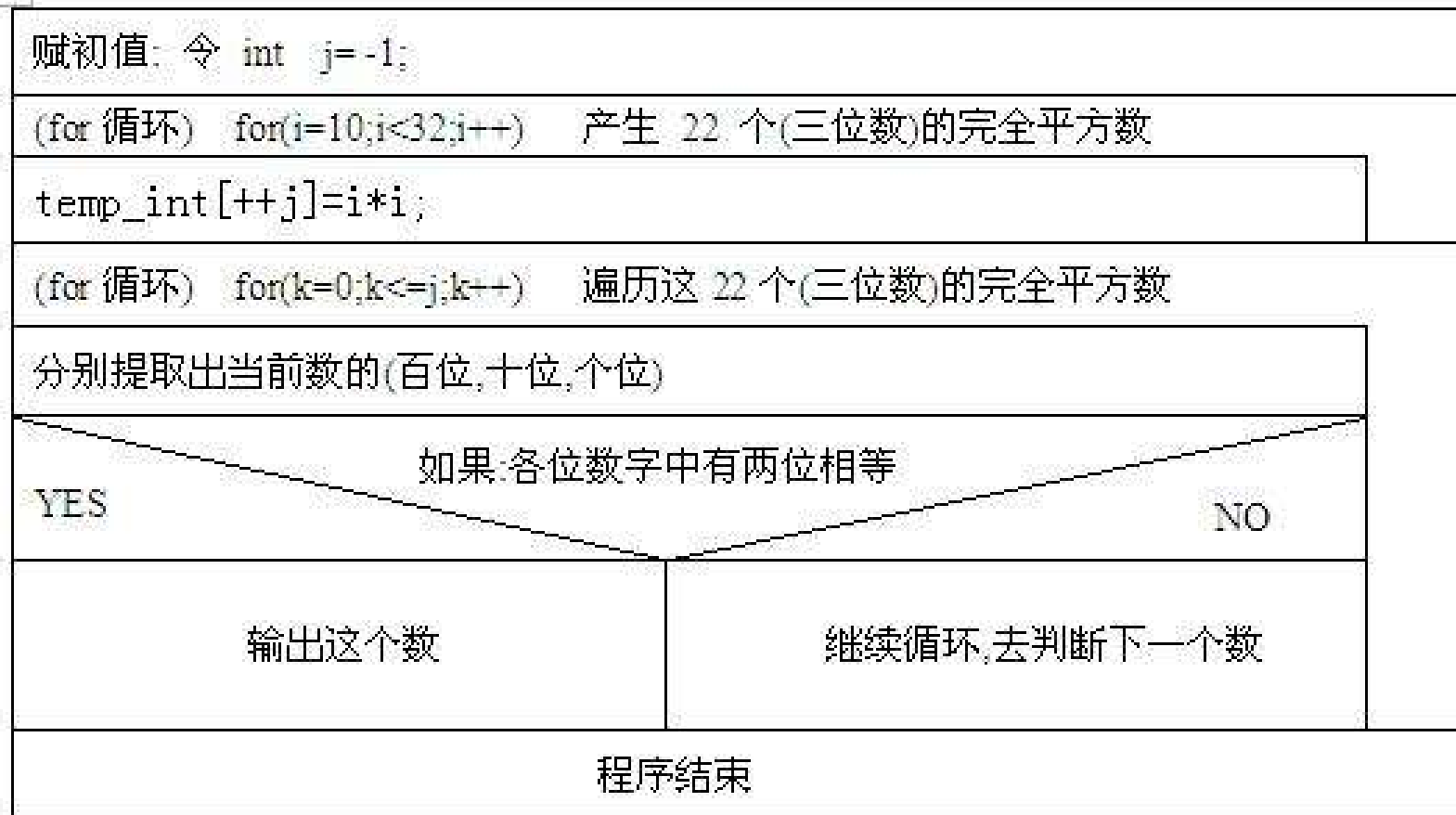
- ✓ PDL

- ✓ IPO 图

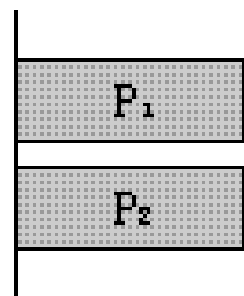
● 程序流程图



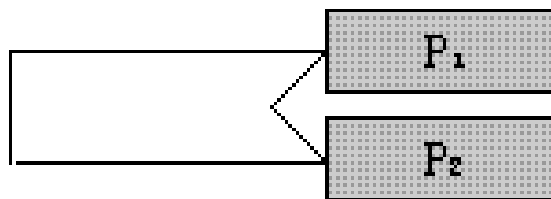
● 盒图



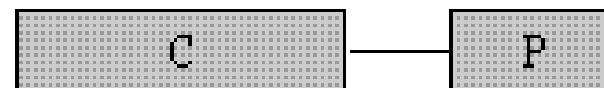
- 问题分析图



顺序



选择



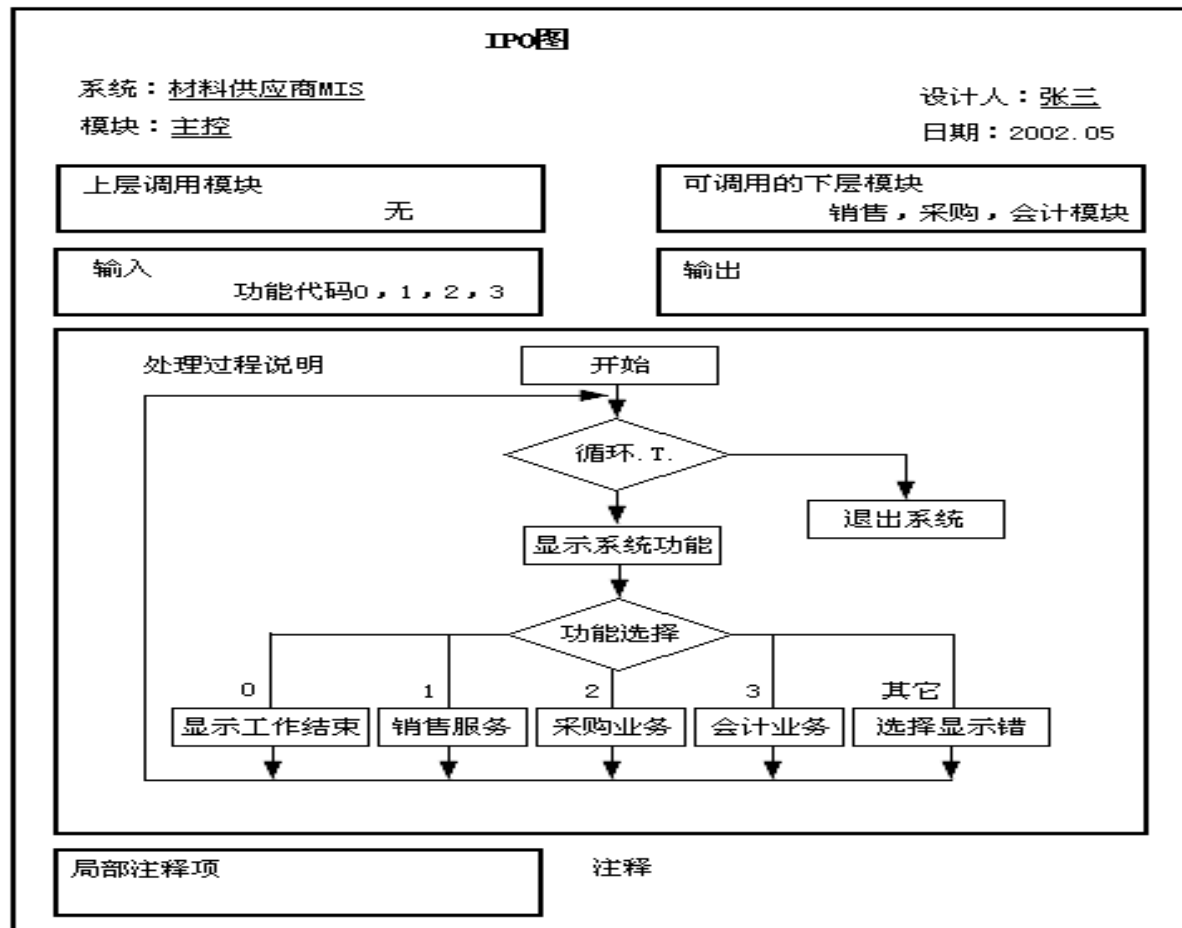
循环

PAD的基本组成

- 4. 过程设计语言

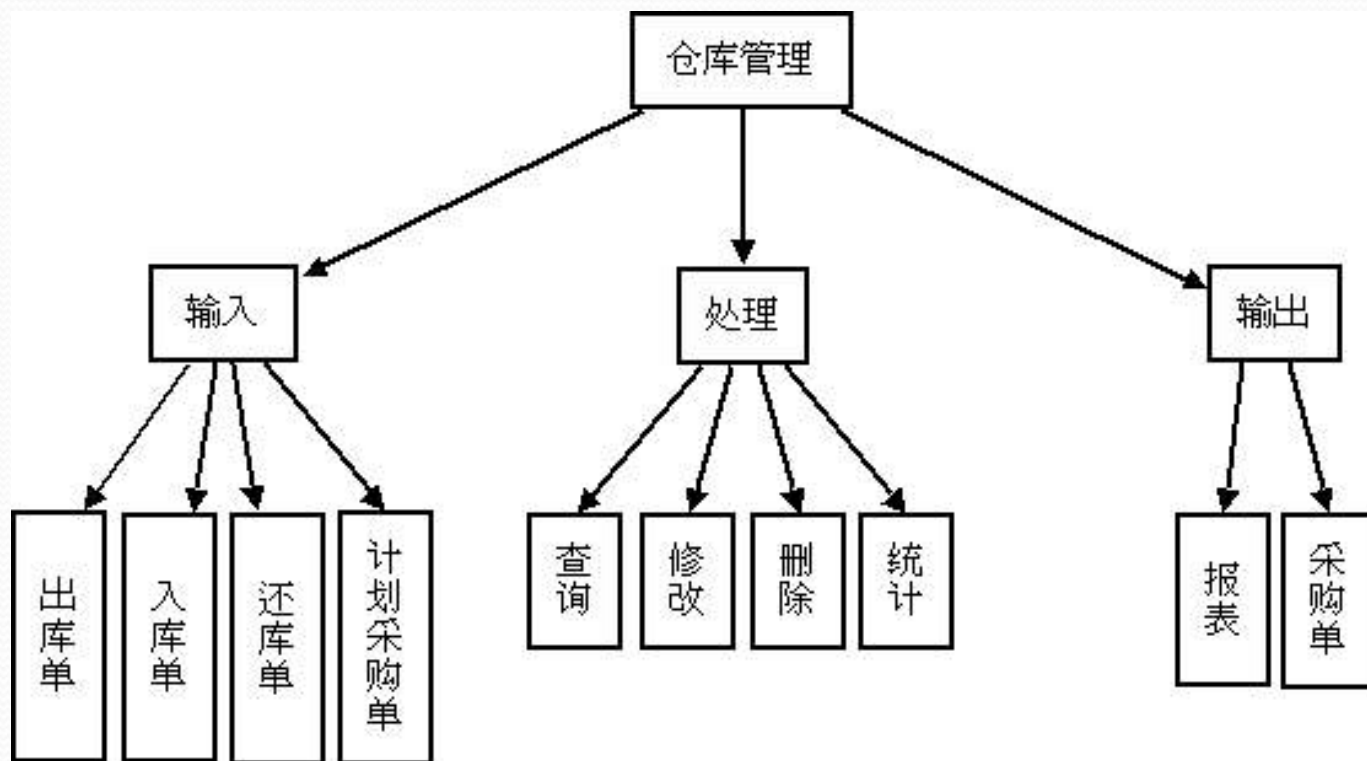
- 过程设计语言(PDL, Process Design Language)是一个笼统的名字, 由许多种不同的过程设计语言。过程设计语言用于描述模块中算法和加工的具体细节, 以便在开发人员之间比较精确的进行交流。

- IPO图
- IPO (Input-Process-Output) 图就是用来表述每个模块的输入，输出和数据加工的重要工具。



HIPO

- H图+IPO图



小结

- ❑ 软件设计的基本目的是解决“系统应该如何实现”这个问题。
- ❑ 模块化和模块独立性
- ❑ 结构化设计主要解决如何将需求分析阶段的**DFD 图推导为软件系统结构**，并用它提供的结构图描述之。
- ❑ 系统设计时要遵循**模块化、自顶向下逐步求精**的基本思想。
- ❑ 详细设计着重对**各个模块的逻辑**进行详细设计。
- ❑ 结构化设计主要解决如何将需求分析阶段的**DFD 图推导为软件系统结构**，并用它提供的结构图描述之。